EN4554 - Deep Learning for Vision

Assignment 2: Convolutional Neural Networks

ManawaduM.D

220381M

GitHub repository:
https://github.com/Demitha-Manawadu/Deep-Learning-for-Vision--EN4554-Assignment-2-on-CNN.git

AI assistance acknowledgment
OpenAI Codex was used to help write and debug the code in this notebook. I have reviewed and understood all of the code and the results.

How to use: Read each task with its saved plots and results. To reproduce it, run cells in order on a Colab T4 and connect Drive. Completed runs load saved checkpoints.

In [1]:
```
from pathlib import Path
import json
import random
import time

import numpy as np
import matplotlib.pyplot as plt
import torch
from torch import nn
from torch.utils.data import DataLoader, Subset
from torchvision import datasets, transforms

SEED = 42
DATA_DIR = Path("data")
try:
    import google.colab
    IN_COLAB = True
except ImportError:
    IN_COLAB = False
if IN_COLAB:
    from google.colab import drive
    drive.mount('/content/drive')
    OUTPUT_DIR = Path('/content/drive/MyDrive/EN4554_220381M/outputs')
else:
    OUTPUT_DIR = Path('outputs')
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
FIGURE_DIR = OUTPUT_DIR / 'figures'
```

```
FIGURE_DIR.mkdir(exist_ok=True)
print('Persistent results/checkpoints directory:', OUTPUT_DIR)
BATCH_SIZE = 128
NUM_WORKERS = 0  # Portable default for Windows notebooks and Colab.
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

def seed_everything(seed=SEED):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False

seed_everything()
print(f"PyTorch: {torch.__version__} | Device: {DEVICE}")
if DEVICE.type == "cuda":
    print(torch.cuda.get_device_name(0))

def atomic_torch_save(payload, path):
    path = Path(path)
    temporary = path.with_name(path.name + '.tmp')
    torch.save(payload, temporary)
    temporary.replace(path)

def atomic_json_save(payload, path):
    path = Path(path)
    temporary = path.with_name(path.name + '.tmp')
    temporary.write_text(json.dumps(payload, indent=2), encoding='utf-8')
    temporary.replace(path)

def capture_rng(loader):
    return {'python': random.getstate(), 'numpy': np.random.get_state(),
            'torch': torch.get_rng_state(),
            'cuda': torch.cuda.get_rng_state_all() if torch.cuda.is_available() els
            'loader': loader.generator.get_state()}

def restore_rng(state, loader):
    random.setstate(state['python'])
    np.random.set_state(state['numpy'])
    torch.set_rng_state(state['torch'].cpu())
    if torch.cuda.is_available() and state['cuda'] is not None:
        if len(state['cuda']) == torch.cuda.device_count():
            torch.cuda.set_rng_state_all([s.cpu() for s in state['cuda']])
    loader.generator.set_state(state['loader'].cpu())

import time as _backup_time
if not hasattr(plt, '_assignment_original_show'):
    plt._assignment_original_show = plt.show

def show_and_save(*args, **kwargs):
    for number in plt.get_fignums():
        figure = plt.figure(number)
        figure.savefig(FIGURE_DIR / f'figure_{_backup_time.time_ns()}_{number}.png'
                       dpi=150, bbox_inches='tight')
```

```
    return plt._assignment_original_show(*args, **kwargs)

plt.show = show_and_save
```

```
Mounted at /content/drive
Persistent results/checkpoints directory: /content/drive/MyDrive/EN4554_220381M/outp
uts
PyTorch: 2.11.0+cu130 | Device: cuda
Tesla T4
```

Data setup

CIFAR-10 is split into 45,000 training and 5,000 validation images using seed 42. Normalization uses only training statistics. Task 3(a) uses a fixed 5,000-image training subset; the test set is used only for Task 4's final evaluation.

In [2]:
```
raw_train = datasets.CIFAR10(root=DATA_DIR, train=True, download=True)

split_generator = torch.Generator().manual_seed(SEED)
indices = torch.randperm(len(raw_train), generator=split_generator)
train_indices = indices[:45000].tolist()
val_indices = indices[45000:].tolist()
small_train_indices = train_indices[:5000]

channel_sum = np.zeros(3, dtype=np.float64)
channel_square_sum = np.zeros(3, dtype=np.float64)
pixel_count = 0
for start in range(0, len(train_indices), 1000):
    pixels = raw_train.data[train_indices[start:start + 1000]].astype(np.float64) /
    channel_sum += pixels.sum(axis=(0, 1, 2))
    channel_square_sum += np.square(pixels).sum(axis=(0, 1, 2))
    pixel_count += pixels.shape[0] * pixels.shape[1] * pixels.shape[2]
mean = channel_sum / pixel_count
std = np.sqrt(np.maximum(channel_square_sum / pixel_count - mean ** 2, 0))
del pixels

normalize = transforms.Normalize(mean.tolist(), std.tolist())
eval_transform = transforms.Compose([transforms.ToTensor(), normalize])
eval_train_dataset = datasets.CIFAR10(
    root=DATA_DIR, train=True, download=False, transform=eval_transform
)
train_dataset = Subset(eval_train_dataset, train_indices)
val_dataset = Subset(eval_train_dataset, val_indices)
small_train_dataset = Subset(eval_train_dataset, small_train_indices)

split_info = {
    "seed": SEED, "train_indices": train_indices,
    "val_indices": val_indices, "small_train_indices": small_train_indices,
    "mean": mean.tolist(), "std": std.tolist(),
}
(OUTPUT_DIR / "data_split.json").write_text(json.dumps(split_info), encoding="utf-8
print(f"Train: {len(train_dataset)} | Validation: {len(val_dataset)}")
print("Training mean:", mean)
print("Training std:", std)
```

```
100%|██████████| 170M/170M [29:35<00:00, 96.0kB/s]
Train: 45000 | Validation: 5000
Training mean: [0.49165571 0.48233405 0.44667923]
Training std: [0.24708116 0.24350869 0.26158066]
```

In [3]:
```python
def make_loader(dataset, shuffle=False, seed=SEED):
    return DataLoader(
        dataset, batch_size=BATCH_SIZE, shuffle=shuffle,
        num_workers=NUM_WORKERS, pin_memory=DEVICE.type == "cuda",
        generator=torch.Generator().manual_seed(seed),
    )

train_loader = make_loader(train_dataset, shuffle=True)
val_loader = make_loader(val_dataset)
small_train_loader = make_loader(small_train_dataset, shuffle=True)

images, labels = next(iter(make_loader(train_dataset)))
fig, axes = plt.subplots(2, 5, figsize=(10, 4))
for ax, image, label in zip(axes.flat, images[:10], labels[:10]):
    display_image = image.permute(1, 2, 0).numpy() * std + mean
    ax.imshow(np.clip(display_image, 0, 1))
    ax.set_title(raw_train.classes[label.item()])
    ax.axis("off")
plt.tight_layout()
plt.show()
```

Data check: The displayed images and labels confirm that the training data loaded correctly. Normalization is reversed only for viewing.

Training and evaluation

The shared helpers train CNNs and measure loss and accuracy. Task 1 uses manual gradients. Every comparison uses a fixed split, seed, and training budget; training accuracy is measured on clean images.

In [4]:
```python
@torch.no_grad()
def evaluate(model, loader):
```

```
    model.eval()
    total_loss, correct, count = 0.0, 0, 0
    for inputs, targets in loader:
        inputs, targets = inputs.to(DEVICE), targets.to(DEVICE)
        logits = model(inputs)
        total_loss += nn.functional.cross_entropy(logits, targets, reduction="sum")
        correct += (logits.argmax(dim=1) == targets).sum().item()
        count += targets.size(0)
    return {"loss": total_loss / count, "accuracy": correct / count}


def train_epoch(model, loader, optimizer):
    model.train()
    total_loss, count = 0.0, 0
    for inputs, targets in loader:
        inputs, targets = inputs.to(DEVICE), targets.to(DEVICE)
        optimizer.zero_grad(set_to_none=True)
        loss = nn.functional.cross_entropy(model(inputs), targets)
        loss.backward()
        optimizer.step()
        total_loss += loss.item() * targets.size(0)
        count += targets.size(0)
    return total_loss / count


def fit(model, optimizer, train_loader, val_loader, epochs):
    model.to(DEVICE)
    history = []
    for epoch in range(1, epochs + 1):
        start = time.perf_counter()
        train_loss = train_epoch(model, train_loader, optimizer)
        validation = evaluate(model, val_loader)
        row = {"epoch": epoch, "train_loss": train_loss,
               "val_loss": validation["loss"], "val_accuracy": validation["accuracy
               "seconds": time.perf_counter() - start}
        history.append(row)
        print(f"Epoch {epoch:02d} | loss {train_loss:.4f} | "
              f"val accuracy {validation['accuracy']:.2%}")
    return history


def plot_history(history, title):
    epochs = [row["epoch"] for row in history]
    fig, axes = plt.subplots(1, 2, figsize=(10, 3))
    axes[0].plot(epochs, [r["train_loss"] for r in history], label="Train")
    axes[0].plot(epochs, [r["val_loss"] for r in history], label="Validation")
    axes[0].set(xlabel="Epoch", ylabel="Cross-entropy loss")
    axes[0].legend()
    axes[1].plot(epochs, [100 * r["val_accuracy"] for r in history])
    axes[1].set(xlabel="Epoch", ylabel="Validation accuracy (%)")
    fig.suptitle(title)
    plt.tight_layout()
    plt.show()


def save_results(name, config, history):
```

```
    payload = {"config": config, "history": history,
               "torch_version": torch.__version__, "seed": SEED}
    (OUTPUT_DIR / f"{name}.json").write_text(
        json.dumps(payload, indent=2), encoding="utf-8"
    )
```

Task 1: Backpropagation

1(a): Manual MLP

Linear, ReLU, and stable softmax cross-entropy are implemented with explicit forward and backward methods. The classifier uses 3,072 inputs, 256 hidden units, and 10 outputs, without autograd for training.

In [5]:
```
class ManualLinear:
    def __init__(self, inputs, outputs, dtype=torch.float32, device="cpu"):
        self.weight = torch.randn(inputs, outputs, dtype=dtype, device=device) * (2
        self.bias = torch.zeros(outputs, dtype=dtype, device=device)
    def forward(self, x):
        self.x = x
        return x @ self.weight + self.bias
    def backward(self, gradient):
        self.dw = self.x.T @ gradient
        self.db = gradient.sum(0)
        return gradient @ self.weight.T

class ManualReLU:
    def forward(self, x):
        self.mask = x > 0
        return x.clamp_min(0)
    def backward(self, gradient):
        return gradient * self.mask

class ManualSigmoid:
    def forward(self, x):
        self.output = torch.sigmoid(x)
        return self.output
    def backward(self, gradient):
        return gradient * self.output * (1 - self.output)

class ManualCrossEntropy:
    def forward(self, logits, labels):
        log_probabilities = logits - torch.logsumexp(logits, dim=1, keepdim=True)
        self.probabilities = log_probabilities.exp()
        self.labels = labels
        return -log_probabilities[torch.arange(len(labels), device=labels.device),
    def backward(self):
        gradient = self.probabilities.clone()
        gradient[torch.arange(len(self.labels), device=self.labels.device), self.la
        return gradient / len(self.labels)

class ManualMLP:
    def __init__(self, widths, activation="relu", dtype=torch.float32, device="cpu"
```

```
        self.layers = []
        for i, (inputs, outputs) in enumerate(zip(widths[:-1], widths[1:])):
            self.layers.append(ManualLinear(inputs, outputs, dtype, device))
            if i < len(widths) - 2:
                self.layers.append(ManualReLU() if activation == "relu" else Manual
        self.criterion = ManualCrossEntropy()
    def forward(self, x):
        for layer in self.layers:
            x = layer.forward(x)
        return x
    def loss(self, x, y):
        return self.criterion.forward(self.forward(x), y)
    def backward(self):
        gradient = self.criterion.backward()
        for layer in reversed(self.layers):
            gradient = layer.backward(gradient)
        return gradient
    def linears(self):
        return [layer for layer in self.layers if isinstance(layer, ManualLinear)]
    def parameters_and_gradients(self):
        return [(p, g) for layer in self.linears()
                for p, g in [(layer.weight, layer.dw), (layer.bias, layer.db)]]

print("Manual layers ready; no autograd is used by these components.")
```

```
Manual layers ready; no autograd is used by these components.
```

1(b): Gradient verification

Manual gradients are checked against autograd and centred finite differences on a small float64 network. The finite-difference step is `1e-6`; `torch.allclose` uses `rtol=1e-5` and `atol=1e-7`. Relative error is `||manual - reference|| / (||manual|| + ||reference||)`.

In [6]:
```
seed_everything()
check_model = ManualMLP([4, 5, 3], dtype=torch.float64)
check_x = torch.randn(3, 4, dtype=torch.float64)
check_y = torch.tensor([0, 1, 2])
manual_loss = check_model.loss(check_x, check_y)
manual_dx = check_model.backward().clone()
pairs = check_model.parameters_and_gradients()
manual_gradients = [g.clone() for _, g in pairs] + [manual_dx]
reference_parameters = [p.clone().requires_grad_(True) for p, _ in pairs]
reference_x = check_x.clone().requires_grad_(True)
hidden = torch.relu(reference_x @ reference_parameters[0] + reference_parameters[1]
reference_loss = nn.functional.cross_entropy(
    hidden @ reference_parameters[2] + reference_parameters[3], check_y)
reference_loss.backward()
autograd_gradients = [p.grad for p in reference_parameters] + [reference_x.grad]

def relative_error(a, b):
    return ((a-b).norm() / (a.norm()+b.norm()).clamp_min(1e-15)).item()

step = 1e-6
```

```
names = ["W1", "b1", "W2", "b2", "input"]
print("Relative error = ||a-b|| / (||a|| + ||b||); h=1e-6")
print(f"{'Tensor':8s} {'Autograd error':>16s} {'Finite-diff error':>18s}  allclose(
for name, tensor, manual, reference in zip(
        names, [p for p, _ in pairs] + [check_x], manual_gradients, autograd_gradie
    numerical = torch.zeros_like(tensor)
    for j in range(tensor.numel()):
        flat = tensor.view(-1)
        original = flat[j].item()
        flat[j] = original + step
        plus = check_model.loss(check_x, check_y).item()
        flat[j] = original - step
        minus = check_model.loss(check_x, check_y).item()
        flat[j] = original
        numerical.view(-1)[j] = (plus - minus) / (2 * step)
    auto_ok = torch.allclose(manual, reference, rtol=1e-5, atol=1e-7)
    fd_ok = torch.allclose(manual, numerical, rtol=1e-5, atol=1e-7)
    print(f"{name:8s} {relative_error(manual, reference):16.3e} "
          f"{relative_error(manual, numerical):18.3e}  {auto_ok}, {fd_ok}")
    assert auto_ok and fd_ok, f"Gradient mismatch: {name}"
```

```
Relative error = ||a-b|| / (||a|| + ||b||); h=1e-6
Tensor     Autograd error  Finite-diff error  allclose(auto, FD)
W1              1.501e-16          2.399e-10  True, True
b1              1.948e-16          3.284e-11  True, True
W2              1.253e-16          5.873e-11  True, True
b2              4.765e-17          1.082e-10  True, True
input           1.106e-16          3.825e-10  True, True
```

Interpretation: All gradient checks pass. Errors are around 10^-16 against autograd and 10^-11 to 10^-10 against finite differences. These very small differences support the correctness of the manual backward calculations.

Autograd checks the manual formulas against PyTorch's derivatives, while finite differences check how the loss changes when a value is slightly perturbed. Finite differences have a larger error because they approximate the derivative using a finite step. Agreement between both checks gives more confidence than either check alone.

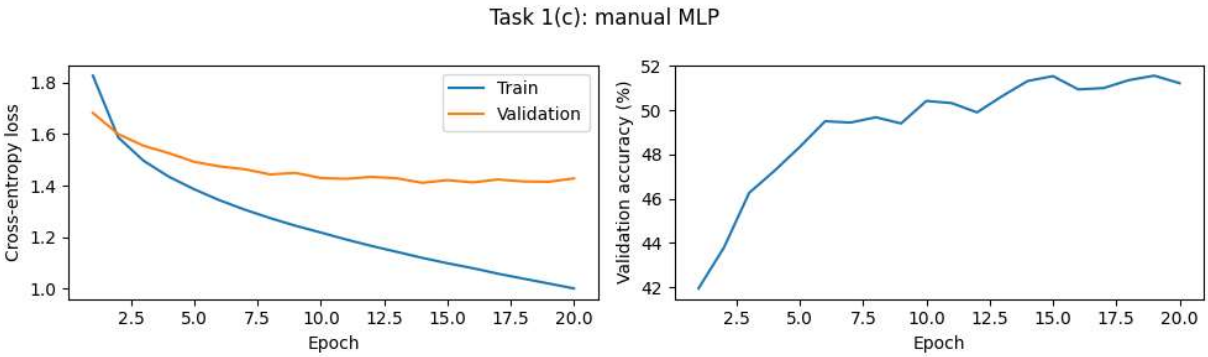
1(c): MLP training

The MLP uses manual SGD with learning rate 0.01, batch size 128, and 20 epochs. Images are flattened before entering the model.

In [7]:
```
seed_everything()
MLP_EPOCHS = 20
MLP_LR = 0.01
manual_model = ManualMLP([3072, 256, 10], device=DEVICE)
manual_train_loader = make_loader(train_dataset, shuffle=True)
manual_history = []

@torch.no_grad()
def evaluate_manual(model, loader):
    loss_sum, correct, count = 0.0, 0, 0
```

```
    for x, y in loader:
        x, y = x.to(DEVICE).flatten(1), y.to(DEVICE)
        logits = model.forward(x)
        loss_sum += nn.functional.cross_entropy(logits, y, reduction="sum").item()
        correct += (logits.argmax(1) == y).sum().item()
        count += len(y)
    return {"loss": loss_sum / count, "accuracy": correct / count}

manual_config = {'widths': [3072, 256, 10], 'lr': MLP_LR, 'epochs': MLP_EPOCHS,
                 'batch_size': BATCH_SIZE, 'seed': SEED,
                 'mean': mean.tolist(), 'std': std.tolist()}
manual_resume_path = OUTPUT_DIR / 'task1_mlp.resume.pt'
if manual_resume_path.exists():
    saved = torch.load(manual_resume_path, map_location='cpu', weights_only=False)
    if saved['config'] == manual_config:
        for layer, state in zip(manual_model.linears(), saved['layers']):
            layer.weight.copy_(state['weight'].to(DEVICE))
            layer.bias.copy_(state['bias'].to(DEVICE))
        manual_history = saved['history']
        restore_rng(saved['rng'], manual_train_loader)
        print(f'Task 1 MLP: resuming after epoch {len(manual_history)}')

for epoch in range(len(manual_history) + 1, MLP_EPOCHS + 1):
    loss_sum, count = 0.0, 0
    for x, y in manual_train_loader:
        x, y = x.to(DEVICE).flatten(1), y.to(DEVICE)
        loss = manual_model.loss(x, y)
        manual_model.backward()
        for parameter, gradient in manual_model.parameters_and_gradients():
            parameter.add_(gradient, alpha=-MLP_LR)
        loss_sum += loss.item() * len(y)
        count += len(y)
    metrics = evaluate_manual(manual_model, val_loader)
    manual_history.append({"epoch": epoch, "train_loss": loss_sum / count,
                           "val_loss": metrics["loss"], "val_accuracy": metrics["ac
    print(f"Epoch {epoch:02d}/{MLP_EPOCHS}: loss={loss_sum/count:.4f}, "
          f"validation accuracy={metrics['accuracy']:.2%}")
    atomic_torch_save({
        'config': manual_config,
        'layers': [{'weight': layer.weight.detach().cpu(), 'bias': layer.bias.detac
                   for layer in manual_model.linears()],
        'history': manual_history, 'rng': capture_rng(manual_train_loader),
    }, manual_resume_path)
    atomic_json_save({'config': manual_config, 'history': manual_history},
                     OUTPUT_DIR / 'task1_mlp.json')
plot_history(manual_history, "Task 1(c): manual MLP")
save_results("task1_mlp", {"widths": [3072, 256, 10], "lr": MLP_LR,
                           "epochs": MLP_EPOCHS, "batch_size": BATCH_SIZE}, manual_
print(f"Final validation accuracy: {manual_history[-1]['val_accuracy']:.2%}")
```

```
Epoch 01/20: loss=1.8270, validation accuracy=41.94%
Epoch 02/20: loss=1.5873, validation accuracy=43.78%
Epoch 03/20: loss=1.4966, validation accuracy=46.26%
Epoch 04/20: loss=1.4345, validation accuracy=47.26%
Epoch 05/20: loss=1.3857, validation accuracy=48.34%
Epoch 06/20: loss=1.3432, validation accuracy=49.50%
Epoch 07/20: loss=1.3064, validation accuracy=49.44%
Epoch 08/20: loss=1.2737, validation accuracy=49.68%
Epoch 09/20: loss=1.2440, validation accuracy=49.40%
Epoch 10/20: loss=1.2177, validation accuracy=50.42%
Epoch 11/20: loss=1.1906, validation accuracy=50.32%
Epoch 12/20: loss=1.1655, validation accuracy=49.90%
Epoch 13/20: loss=1.1426, validation accuracy=50.64%
Epoch 14/20: loss=1.1193, validation accuracy=51.32%
Epoch 15/20: loss=1.0983, validation accuracy=51.54%
Epoch 16/20: loss=1.0792, validation accuracy=50.94%
Epoch 17/20: loss=1.0576, validation accuracy=51.00%
Epoch 18/20: loss=1.0385, validation accuracy=51.36%
Epoch 19/20: loss=1.0195, validation accuracy=51.56%
Epoch 20/20: loss=1.0004, validation accuracy=51.22%
```




```
Final validation accuracy: 51.22%
```

Interpretation: Training loss falls from **1.8270 to 1.0004**. Final validation accuracy is **51.22%**, with a best observed value of **51.56% at epoch 19**. Validation improvement slows while training loss keeps falling, so later training gives limited generalization gains. The MLP does not directly use local image structure.

Flattening treats the image as a long list of pixels, so the model cannot directly share a filter across different image positions. The small difference between the best and final validation accuracy also shows that extra epochs do not always improve unseen-image performance. A CNN is better suited to testing whether spatial features improve classification.

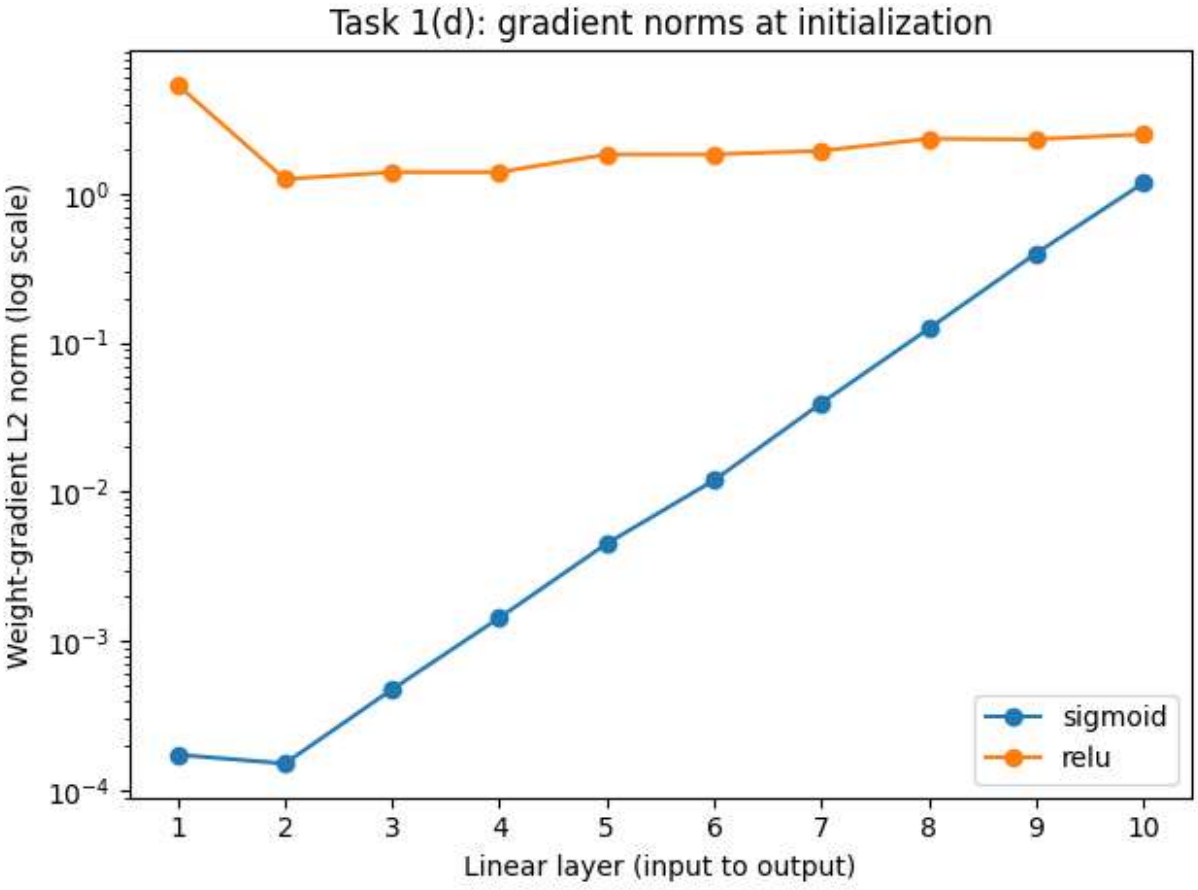
1(d): Gradient flow

A ten-linear-layer MLP compares sigmoid and ReLU on the same batch, with identical Kaiming-initialized weights. Nine layers are hidden layers and one is the output layer.

In [8]:
```
seed_everything()
gradient_x, gradient_y = next(iter(make_loader(train_dataset)))
gradient_x = gradient_x.to(DEVICE).flatten(1)
gradient_y = gradient_y.to(DEVICE)
```

```python
widths = [3072] + [128] * 9 + [10]
gradient_results = {}
for activation in ["sigmoid", "relu"]:
    seed_everything()
    network = ManualMLP(widths, activation=activation, device=DEVICE)
    network.loss(gradient_x, gradient_y)
    network.backward()
    norms = [layer.dw.norm().item() for layer in network.linears()]
    gradient_results[activation] = norms
    print(activation, "weight-gradient norms:", [f"{v:.3e}" for v in norms])
    plt.semilogy(range(1, 11), np.maximum(norms, 1e-30), marker="o", label=activati
plt.xticks(range(1, 11))
plt.xlabel("Linear layer (input to output)")
plt.ylabel("Weight-gradient L2 norm (log scale)")
plt.title("Task 1(d): gradient norms at initialization")
plt.legend()
plt.tight_layout()
plt.show()
(OUTPUT_DIR / "task1_gradient_norms.json").write_text(json.dumps(gradient_results,
```

```
sigmoid weight-gradient norms: ['1.733e-04', '1.511e-04', '4.753e-04', '1.436e-03',
'4.511e-03', '1.199e-02', '3.951e-02', '1.245e-01', '3.976e-01', '1.180e+00']
relu weight-gradient norms: ['5.383e+00', '1.253e+00', '1.397e+00', '1.398e+00', '1.
837e+00', '1.839e+00', '1.939e+00', '2.341e+00', '2.317e+00', '2.505e+00']
```



Out[8]: 531

Interpretation: Sigmoid's gradient norm falls from **1.180** at the output layer to **0.0001733** at the first layer, showing vanishing gradients. ReLU keeps larger gradients (**2.505** at the

output and **5.383** at the first layer). Repeated small sigmoid derivatives weaken backward signals. This comparison uses Kaiming initialization, which is designed for ReLU.

Early layers need a useful gradient to learn their weights. When that signal becomes very small, these layers update slowly even if later layers receive larger gradients. The log-scale axis makes these large differences in gradient size visible. These measurements are at initialization, rather than evidence of accuracy after training.

Discussion points

- Agreement with two gradient checks supports the manual implementation before training.
- The MLP learns useful patterns, but its validation improvement levels off near 51%.
- ReLU passes stronger gradients to early layers in this initialization test; this helps explain why activation choice matters in deep networks.

Task 2: Optimizers

2(a): Custom momentum SGD and Adam

Both optimizers inherit from `torch.optim.Optimizer`. Momentum stores a running gradient velocity. Adam stores gradient and squared-gradient averages with bias correction. Ten updates are checked against PyTorch, including weight decay and parameter groups.

In [9]:
```
import math
import copy
import hashlib
from IPython.display import display

class MomentumSGD(torch.optim.Optimizer):
    """SGD with classical momentum, optional coupled weight decay, no dampening."""
    def __init__(self, params, lr=0.01, momentum=0.9, weight_decay=0.0):
        if lr < 0 or not 0 <= momentum < 1 or weight_decay < 0:
            raise ValueError("Invalid optimizer hyperparameters")
        super().__init__(params, dict(lr=lr, momentum=momentum, weight_decay=weight

    @torch.no_grad()
    def step(self, closure=None):
        loss = None
        if closure is not None:
            with torch.enable_grad():
                loss = closure()
        for group in self.param_groups:
            for p in group['params']:
                if p.grad is None:
                    continue
                if p.grad.is_sparse:
                    raise RuntimeError('Sparse gradients are unsupported')
                gradient = p.grad
```

```
                if group['weight_decay']:
                    gradient = gradient.add(p, alpha=group['weight_decay'])
                if group['momentum']:
                    state = self.state[p]
                    if 'velocity' not in state:
                        state['velocity'] = gradient.clone()
                    else:
                        state['velocity'].mul_(group['momentum']).add_(gradient)
                    gradient = state['velocity']
                p.add_(gradient, alpha=-group['lr'])
        return loss

class CustomAdam(torch.optim.Optimizer):
    """Adam with bias correction and optional coupled L2 weight decay (not AdamW)."
    def __init__(self, params, lr=0.001, betas=(0.9, 0.999), eps=1e-8, weight_decay
        if lr < 0 or eps < 0 or weight_decay < 0 or any(not 0 <= b < 1 for b in bet
            raise ValueError('Invalid optimizer hyperparameters')
        super().__init__(params, dict(lr=lr, betas=betas, eps=eps, weight_decay=wei

    @torch.no_grad()
    def step(self, closure=None):
        loss = None
        if closure is not None:
            with torch.enable_grad():
                loss = closure()
        for group in self.param_groups:
            beta1, beta2 = group['betas']
            for p in group['params']:
                if p.grad is None:
                    continue
                if p.grad.is_sparse:
                    raise RuntimeError('Sparse gradients are unsupported')
                gradient = p.grad
                if group['weight_decay']:
                    gradient = gradient.add(p, alpha=group['weight_decay'])
                state = self.state[p]
                if not state:
                    state.update(step=0, m=torch.zeros_like(p), v=torch.zeros_like(
                state['step'] += 1
                state['m'].mul_(beta1).add_(gradient, alpha=1-beta1)
                state['v'].mul_(beta2).addcmul_(gradient, gradient, value=1-beta2)
                m_hat = state['m'] / (1-beta1**state['step'])
                v_hat = state['v'] / (1-beta2**state['step'])
                p.addcdiv_(m_hat, v_hat.sqrt().add_(group['eps']), value=-group['lr
        return loss

optimizer_check_rows = []
for weight_decay in (0.0, 0.01):
    for name, implementation, reference in [
        ('Momentum', MomentumSGD, torch.optim.SGD),
        ('Adam', CustomAdam, torch.optim.Adam),
    ]:
        seed_everything()
        parameters = [nn.Parameter(torch.randn(3, 4, dtype=torch.float64)),
                      nn.Parameter(torch.randn(4, dtype=torch.float64))]
        references = [nn.Parameter(p.detach().clone()) for p in parameters]
```

```
        kwargs = {'lr': 0.007, 'weight_decay': weight_decay}
        kwargs.update({'momentum': 0.85} if name == 'Momentum' else {'betas': (0.8,
        opt = implementation([{'params': [parameters[0]]},
                              {'params': [parameters[1]], 'lr': 0.003}], **kwargs)
        ref = reference([{'params': [references[0]]},
                         {'params': [references[1]], 'lr': 0.003}], foreach=False,
        max_error = 0.0
        matched = True
        for step in range(10):
            for j, (p, q) in enumerate(zip(parameters, references)):
                gradient = None if step == 4 and j == 1 else torch.randn_like(p)
                p.grad = gradient
                q.grad = None if gradient is None else gradient.clone()
            opt.step()
            ref.step()
            for p, q in zip(parameters, references):
                max_error = max(max_error, (p-q).abs().max().item())
                matched &= torch.allclose(p, q, rtol=1e-10, atol=1e-12)
        optimizer_check_rows.append([name, weight_decay, max_error, matched])
        assert matched, f'{name} did not match PyTorch'
print('Optimizer | weight decay | max absolute parameter error | allclose')
for row in optimizer_check_rows:
    print(f'{row[0]:8s} | {row[1]:.2f} | {row[2]:.3e} | {row[3]}')
```

```
Optimizer | weight decay | max absolute parameter error | allclose
Momentum | 0.00 | 0.000e+00 | True
Adam     | 0.00 | 5.551e-17 | True
Momentum | 0.01 | 0.000e+00 | True
Adam     | 0.01 | 0.000e+00 | True
```

Interpretation: Both custom optimizers match PyTorch in all checks. Momentum's maximum parameter error is **zero**; Adam's largest error is **5.551e-17**. The tiny Adam difference is floating-point rounding.

The checks cover several consecutive updates, which is important because both optimizers depend on stored information from earlier steps. Adam's bias correction compensates for moving averages that start at zero. Matching these tests supports using the custom optimizers in the later experiments, although it does not test every possible PyTorch optimizer option.

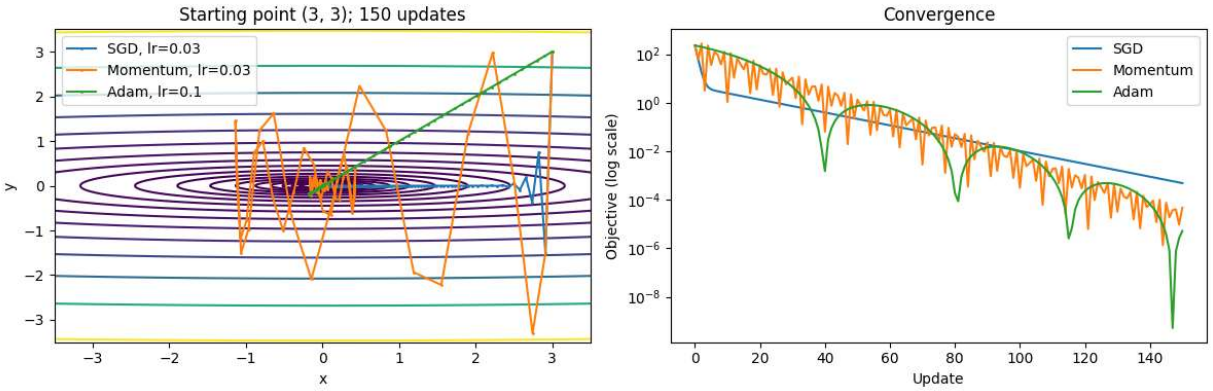
2(b): Optimizer trajectories

SGD, momentum, and Adam minimize `f(x,y) = 0.5(x**2 + 50*y**2)` from (3, 3) for 150 updates. Learning rates are 0.03 for SGD/momentum and 0.1 for Adam.

In [10]:
```
quadratic_paths = {}
quadratic_settings = {'SGD': 0.03, 'Momentum': 0.03, 'Adam': 0.1}
for name, lr in quadratic_settings.items():
    position = nn.Parameter(torch.tensor([3.0, 3.0], dtype=torch.float64))
    optimizer = (torch.optim.SGD([position], lr=lr) if name == 'SGD' else
                 MomentumSGD([position], lr=lr, momentum=0.9) if name == 'Momentum'
                 CustomAdam([position], lr=lr))
    path = [position.detach().numpy().copy()]
```

```
    for _ in range(150):
        optimizer.zero_grad(set_to_none=True)
        objective = 0.5 * (position[0]**2 + 50*position[1]**2)
        objective.backward()
        optimizer.step()
        path.append(position.detach().numpy().copy())
    quadratic_paths[name] = np.asarray(path)

xx, yy = np.meshgrid(np.linspace(-3.5, 3.5, 300), np.linspace(-3.5, 3.5, 300))
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].contour(xx, yy, 0.5*(xx**2+50*yy**2), levels=np.geomspace(0.05, 300, 18))
for name, path in quadratic_paths.items():
    axes[0].plot(path[:, 0], path[:, 1], '.-', markersize=2,
                 label=f'{name}, lr={quadratic_settings[name]}')
    values = 0.5*(path[:, 0]**2 + 50*path[:, 1]**2)
    axes[1].semilogy(np.maximum(values, 1e-20), label=name)
    print(f'{name}: final objective={values[-1]:.6g}, position={path[-1]}')
axes[0].set(xlabel='x', ylabel='y', title='Starting point (3, 3); 150 updates')
axes[1].set(xlabel='Update', ylabel='Objective (log scale)', title='Convergence')
for ax in axes:
    ax.legend()
plt.tight_layout()
plt.show()
```

```
SGD: final objective=0.000483875, position=[3.11086665e-02 2.10194770e-45]
Momentum: final objective=4.70575e-05, position=[ 0.00112154 -0.00136277]
Adam: final objective=5.17242e-06, position=[0.00045038 0.00045038]
```



Interpretation: The bowl is steeper along y, making progress uneven across directions. Final objective values are **4.839e-4 for SGD**, **4.706e-5 for momentum**, and **5.172e-6 for Adam**. Momentum uses gradient history, while Adam adapts update scales. Adam finishes lowest here, but it uses a different learning rate, so this is not a universal ranking.

A smaller final objective means the position is closer to the minimum at (0, 0). Plain SGD must use a step size that remains stable along the steep direction, which can slow movement along the flatter direction. The trajectory and objective plots therefore show complementary views: the route taken and the speed of reducing the objective.

2(c): Learning-rate sensitivity

The same 94,538-parameter CNN is trained for ten epochs at three learning rates per optimizer. The best rate is selected using final validation accuracy.

In [11]:

```
EPOCHS_T2 = 10
EPOCHS_T3A = 30
EPOCHS_T3B = 30
EPOCHS_T4 = 20
REUSE_SAVED_RUNS = True
EXPERIMENT_VERSION = '220381M-v1'

class SmallCNN(nn.Module):
    def __init__(self, batch_norm=False, dropout=0.0):
        super().__init__()
        layers = []
        channels = 3
        for width in (32, 64, 128):
            layers.append(nn.Conv2d(channels, width, 3, padding=1, bias=not batch_n
            if batch_norm:
                layers.append(nn.BatchNorm2d(width))
            layers.extend([nn.ReLU(), nn.MaxPool2d(2)])
            channels = width
        self.features = nn.Sequential(*layers)
        self.head = nn.Sequential(nn.AdaptiveAvgPool2d(1), nn.Flatten(),
                                  nn.Dropout(dropout), nn.Linear(128, 10))
        initialize_network(self)
    def forward(self, x):
        return self.head(self.features(x))

def initialize_network(model):
    for layer in model.modules():
        if isinstance(layer, nn.Conv2d):
            nn.init.kaiming_normal_(layer.weight, mode='fan_in', nonlinearity='relu
            if layer.bias is not None:
                nn.init.zeros_(layer.bias)
        elif isinstance(layer, nn.BatchNorm2d):
            nn.init.ones_(layer.weight)
            nn.init.zeros_(layer.bias)
        elif isinstance(layer, nn.Linear):
            nn.init.kaiming_normal_(layer.weight, nonlinearity='linear')
            nn.init.zeros_(layer.bias)

def make_optimizer(name, parameters, lr, weight_decay=0.0):
    if name == 'SGD':
        return torch.optim.SGD(parameters, lr=lr, weight_decay=weight_decay)
    if name == 'Momentum':
        return MomentumSGD(parameters, lr=lr, momentum=0.9, weight_decay=weight_dec
    if name == 'Adam':
        return CustomAdam(parameters, lr=lr, weight_decay=weight_decay)
    raise ValueError(name)

def show_table(rows, columns):
    from html import escape
    header = ''.join(f'<th>{escape(str(c))}</th>' for c in columns)
    body = ''.join('<tr>' + ''.join(f'<td>{escape(str(v))}</td>' for v in row) + '<
                   for row in rows)
```

```
    display(__import__('IPython').display.HTML(
        '<table><thead><tr>'+header+'</tr></thead><tbody>'+body+'</tbody></table>'))

def run_experiment(tag, model_factory, config, training_dataset=train_dataset,
                   clean_dataset=train_dataset, augment_batch=None, mixup_alpha=0.0
    """Save every completed epoch and resume interrupted runs; never select using t
    signature = {**config, 'seed': SEED, 'batch_size': BATCH_SIZE,
                 'train_size': len(training_dataset), 'version': EXPERIMENT_VERSION
                 'mean': mean.tolist(), 'std': std.tolist()}
    digest = hashlib.sha256(json.dumps(signature, sort_keys=True).encode()).hexdige
    stem = OUTPUT_DIR / f'{tag}_{digest}'
    result_path = Path(str(stem) + '.json')
    checkpoint_path = Path(str(stem) + '.pt')
    resume_path = Path(str(stem) + '.resume.pt')
    if REUSE_SAVED_RUNS and result_path.exists() and checkpoint_path.exists():
        result = json.loads(result_path.read_text())
        if result['config'] == signature and result['status'] == 'complete':
            print(f'{tag}: loaded saved run')
            return result
    seed_everything()
    model = model_factory().to(DEVICE)
    optimizer = make_optimizer(config['optimizer'], model.parameters(), config['lr'
                               config.get('weight_decay', 0.0))
    loader = make_loader(training_dataset, shuffle=True)
    clean_loader = make_loader(clean_dataset)
    scheduler = (torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, config['epoc
                 if config.get('cosine', False) else None)
    history = []
    elapsed_before = 0.0
    if REUSE_SAVED_RUNS and resume_path.exists():
        saved = torch.load(resume_path, map_location='cpu', weights_only=False)
        if saved['config'] == signature:
            model.load_state_dict(saved['model'])
            optimizer.load_state_dict(saved['optimizer'])
            if scheduler is not None:
                scheduler.load_state_dict(saved['scheduler'])
            history = saved['history']
            elapsed_before = saved['seconds']
            restore_rng(saved['rng'], loader)
            print(f'{tag}: resuming after epoch {len(history)}')
    started = time.perf_counter()
    status = 'complete'
    for epoch in range(len(history) + 1, config['epochs'] + 1):
        model.train()
        loss_sum, seen = 0.0, 0
        for x, y in loader:
            x, y = x.to(DEVICE), y.to(DEVICE)
            if augment_batch is not None:
                x = augment_batch(x)
            optimizer.zero_grad(set_to_none=True)
            if mixup_alpha > 0:
                x, targets_a, targets_b, lam = mixup_batch(x, y, mixup_alpha)
                logits = model(x)
                loss = (lam * nn.functional.cross_entropy(logits, targets_a) +
                        (1-lam) * nn.functional.cross_entropy(logits, targets_b))
            else:
```

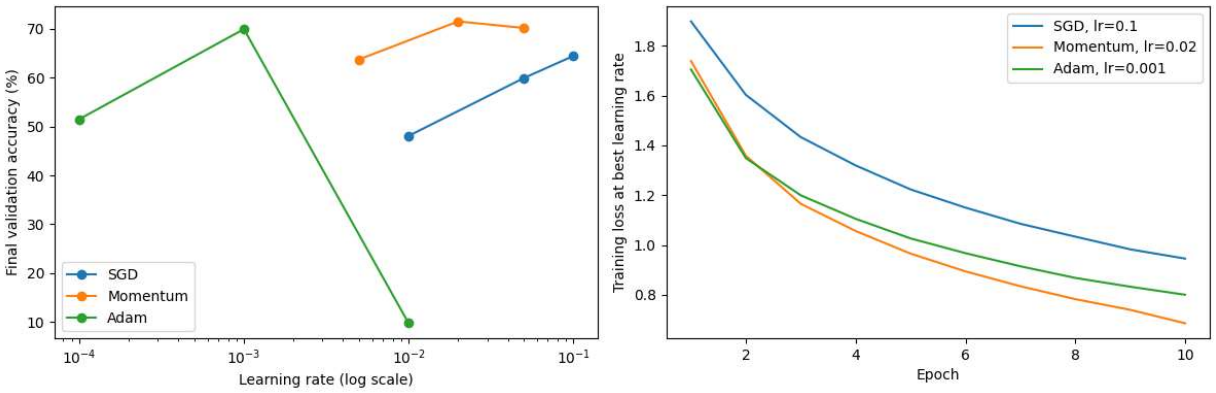
```
                loss = nn.functional.cross_entropy(model(x), y)
            if not torch.isfinite(loss):
                status = 'diverged'
                break
            loss.backward()
            optimizer.step()
            loss_sum += loss.item() * len(y)
            seen += len(y)
        if status != 'complete' or not all(torch.isfinite(p).all() for p in model.p
            status = 'diverged'
            print(f'{tag}: non-finite loss/parameters at epoch {epoch}; excluded fr
            break
        validation = evaluate(model, val_loader)
        clean_train = evaluate(model, clean_loader)
        if not all(math.isfinite(v) for v in [validation['loss'], clean_train['loss
            status = 'diverged'
            break
        history.append({'epoch': epoch, 'train_loss': loss_sum/seen,
                        'clean_train_loss': clean_train['loss'],
                        'train_accuracy': clean_train['accuracy'],
                        'val_loss': validation['loss'], 'val_accuracy': validation[
        print(f"{tag} {epoch:02d}/{config['epochs']}: loss={loss_sum/seen:.4f}, "
              f"train={clean_train['accuracy']:.2%}, val={validation['accuracy']:.2
        if scheduler is not None:
            scheduler.step()
        atomic_torch_save({
            'config': signature, 'model': model.state_dict(),
            'optimizer': optimizer.state_dict(),
            'scheduler': scheduler.state_dict() if scheduler is not None else None,
            'history': history, 'seconds': elapsed_before + time.perf_counter()-sta
            'rng': capture_rng(loader),
        }, resume_path)
        atomic_json_save({'config': signature, 'history': history,
                          'status': 'in_progress'}, result_path)
    result = {'config': signature, 'history': history, 'status': status,
              'parameters': sum(p.numel() for p in model.parameters()),
              'seconds': elapsed_before + time.perf_counter()-started, 'checkpoint'
    if status == 'complete':
        atomic_torch_save({k: v.detach().cpu() for k, v in model.state_dict().items
    atomic_json_save(result, result_path)
    return result

def comparison_table(results):
    rows = []
    for name, run in results.items():
        if run['status'] == 'complete':
            r = run['history'][-1]
            rows.append([name, f"{100*r['train_accuracy']:.2f}",
                        f"{100*r['val_accuracy']:.2f}",
                        f"{100*(r['train_accuracy']-r['val_accuracy']):.2f}",
                        f"{r['train_loss']:.4f}", run['parameters'], f"{run['secon
        else:
            rows.append([name, 'DIVERGED', '-', '-', '-', run['parameters'], '-'])
    show_table(rows, ['Configuration', 'Clean train (%)', 'Validation (%)',
                      'Gap (pp)', 'Training loss', 'Parameters', 'Minutes'])
```

```
def plot_comparison(results, title):
    fig, axes = plt.subplots(1, 2, figsize=(13, 4))
    for name, run in results.items():
        h = run['history']
        if not h:
            continue
        epochs = [r['epoch'] for r in h]
        axes[0].plot(epochs, [r['train_loss'] for r in h], label=name)
        line, = axes[1].plot(epochs, [100*r['val_accuracy'] for r in h], label=name
        axes[1].plot(epochs, [100*r['train_accuracy'] for r in h], '--',
                     color=line.get_color(), alpha=0.6, label=name+' train')
    axes[0].set(xlabel='Epoch', ylabel='Training objective')
    axes[1].set(xlabel='Epoch', ylabel='Accuracy (%)')
    for ax in axes:
        ax.legend(fontsize=7)
    fig.suptitle(title)
    plt.tight_layout()
    plt.show()

LEARNING_RATES = {'SGD': [0.01, 0.05, 0.1],
                  'Momentum': [0.005, 0.02, 0.05],
                  'Adam': [0.0001, 0.001, 0.01]}
task2_results = {}
for name, rates in LEARNING_RATES.items():
    for lr in rates:
        tag = f'task2_{name}_{lr}'
        config = {'architecture': 'SmallCNN_noBN', 'optimizer': name, 'lr': lr,
                  'epochs': EPOCHS_T2, 'selection': 'final_validation_accuracy'}
        task2_results[(name, lr)] = run_experiment(tag, SmallCNN, config)
comparison_table({f'{name}, lr={lr}': r for (name, lr), r in task2_results.items()}
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
task2_best = {}
for name, rates in LEARNING_RATES.items():
    eligible = [(lr, task2_results[(name, lr)]) for lr in rates
                if task2_results[(name, lr)]['status'] == 'complete']
    axes[0].semilogx([lr for lr, _ in eligible],
                     [100*r['history'][-1]['val_accuracy'] for _, r in eligible], 'o
    if eligible:
        best_lr, best = max(eligible, key=lambda item: item[1]['history'][-1]['val_
        task2_best[name] = best_lr
        axes[1].plot([r['epoch'] for r in best['history']],
                     [r['train_loss'] for r in best['history']], label=f'{name}, lr
axes[0].set(xlabel='Learning rate (log scale)', ylabel='Final validation accuracy (
axes[1].set(xlabel='Epoch', ylabel='Training loss at best learning rate')
for ax in axes:
    ax.legend()
plt.tight_layout()
plt.show()
print('Best rates selected using final validation accuracy:', task2_best)
```

```
task2_SGD_0.01 01/10: loss=2.0024, train=30.18%, val=29.42%
task2_SGD_0.01 02/10: loss=1.8034, train=37.55%, val=36.80%
task2_SGD_0.01 03/10: loss=1.7034, train=35.89%, val=35.02%
task2_SGD_0.01 04/10: loss=1.6386, train=41.38%, val=40.24%
task2_SGD_0.01 05/10: loss=1.5813, train=43.63%, val=42.82%
task2_SGD_0.01 06/10: loss=1.5406, train=42.98%, val=42.46%
task2_SGD_0.01 07/10: loss=1.4980, train=46.68%, val=45.60%
task2_SGD_0.01 08/10: loss=1.4624, train=47.36%, val=46.24%
task2_SGD_0.01 09/10: loss=1.4328, train=45.17%, val=44.04%
task2_SGD_0.01 10/10: loss=1.4083, train=49.52%, val=48.08%
task2_SGD_0.05 01/10: loss=1.9076, train=33.57%, val=32.50%
task2_SGD_0.05 02/10: loss=1.6512, train=43.09%, val=42.48%
task2_SGD_0.05 03/10: loss=1.5135, train=48.07%, val=47.00%
task2_SGD_0.05 04/10: loss=1.4185, train=49.88%, val=48.22%
task2_SGD_0.05 05/10: loss=1.3339, train=51.32%, val=50.26%
task2_SGD_0.05 06/10: loss=1.2714, train=52.76%, val=50.46%
task2_SGD_0.05 07/10: loss=1.2119, train=57.24%, val=54.96%
task2_SGD_0.05 08/10: loss=1.1626, train=58.43%, val=56.62%
task2_SGD_0.05 09/10: loss=1.1168, train=55.81%, val=54.32%
task2_SGD_0.05 10/10: loss=1.0852, train=61.77%, val=59.90%
task2_SGD_0.1 01/10: loss=1.8975, train=34.89%, val=33.84%
task2_SGD_0.1 02/10: loss=1.6026, train=46.53%, val=46.08%
task2_SGD_0.1 03/10: loss=1.4330, train=53.19%, val=51.86%
task2_SGD_0.1 04/10: loss=1.3193, train=54.26%, val=52.84%
task2_SGD_0.1 05/10: loss=1.2231, train=55.31%, val=53.54%
task2_SGD_0.1 06/10: loss=1.1500, train=59.99%, val=57.82%
task2_SGD_0.1 07/10: loss=1.0851, train=63.95%, val=61.28%
task2_SGD_0.1 08/10: loss=1.0336, train=63.36%, val=61.26%
task2_SGD_0.1 09/10: loss=0.9825, train=64.08%, val=61.98%
task2_SGD_0.1 10/10: loss=0.9454, train=66.56%, val=64.42%
task2_Momentum_0.005 01/10: loss=1.8428, train=40.10%, val=39.70%
task2_Momentum_0.005 02/10: loss=1.5435, train=49.54%, val=49.20%
task2_Momentum_0.005 03/10: loss=1.4064, train=54.30%, val=53.34%
task2_Momentum_0.005 04/10: loss=1.3175, train=56.62%, val=55.30%
task2_Momentum_0.005 05/10: loss=1.2342, train=58.69%, val=57.30%
task2_Momentum_0.005 06/10: loss=1.1751, train=60.66%, val=58.54%
task2_Momentum_0.005 07/10: loss=1.1207, train=62.50%, val=60.50%
task2_Momentum_0.005 08/10: loss=1.0707, train=65.27%, val=62.94%
task2_Momentum_0.005 09/10: loss=1.0299, train=65.68%, val=63.00%
task2_Momentum_0.005 10/10: loss=0.9952, train=66.14%, val=63.72%
task2_Momentum_0.02 01/10: loss=1.7386, train=43.06%, val=42.98%
task2_Momentum_0.02 02/10: loss=1.3581, train=57.23%, val=56.92%
task2_Momentum_0.02 03/10: loss=1.1653, train=62.48%, val=61.14%
task2_Momentum_0.02 04/10: loss=1.0561, train=64.23%, val=62.58%
task2_Momentum_0.02 05/10: loss=0.9653, train=68.38%, val=65.58%
task2_Momentum_0.02 06/10: loss=0.8941, train=71.08%, val=67.46%
task2_Momentum_0.02 07/10: loss=0.8343, train=73.53%, val=69.36%
task2_Momentum_0.02 08/10: loss=0.7826, train=73.94%, val=69.40%
task2_Momentum_0.02 09/10: loss=0.7397, train=75.22%, val=69.80%
task2_Momentum_0.02 10/10: loss=0.6858, train=78.23%, val=71.50%
task2_Momentum_0.05 01/10: loss=1.7533, train=44.69%, val=44.22%
task2_Momentum_0.05 02/10: loss=1.3389, train=57.87%, val=56.90%
task2_Momentum_0.05 03/10: loss=1.1532, train=62.74%, val=61.58%
task2_Momentum_0.05 04/10: loss=1.0420, train=65.76%, val=62.70%
task2_Momentum_0.05 05/10: loss=0.9725, train=69.66%, val=66.60%
task2_Momentum_0.05 06/10: loss=0.9031, train=70.63%, val=66.64%
```

```
task2_Momentum_0.05 07/10: loss=0.8540, train=72.16%, val=68.10%
task2_Momentum_0.05 08/10: loss=0.8066, train=74.45%, val=69.42%
task2_Momentum_0.05 09/10: loss=0.7611, train=74.79%, val=68.46%
task2_Momentum_0.05 10/10: loss=0.7261, train=76.95%, val=70.16%
task2_Adam_0.0001 01/10: loss=1.9756, train=36.24%, val=35.60%
task2_Adam_0.0001 02/10: loss=1.7242, train=40.69%, val=40.70%
task2_Adam_0.0001 03/10: loss=1.6251, train=43.71%, val=42.62%
task2_Adam_0.0001 04/10: loss=1.5606, train=46.47%, val=45.50%
task2_Adam_0.0001 05/10: loss=1.5099, train=47.92%, val=47.12%
task2_Adam_0.0001 06/10: loss=1.4733, train=47.42%, val=46.48%
task2_Adam_0.0001 07/10: loss=1.4391, train=50.40%, val=49.64%
task2_Adam_0.0001 08/10: loss=1.4079, train=50.19%, val=49.14%
task2_Adam_0.0001 09/10: loss=1.3815, train=52.89%, val=51.18%
task2_Adam_0.0001 10/10: loss=1.3542, train=52.75%, val=51.50%
task2_Adam_0.001 01/10: loss=1.7047, train=48.03%, val=47.52%
task2_Adam_0.001 02/10: loss=1.3486, train=57.19%, val=56.68%
task2_Adam_0.001 03/10: loss=1.1993, train=61.84%, val=59.96%
task2_Adam_0.001 04/10: loss=1.1050, train=63.78%, val=61.74%
task2_Adam_0.001 05/10: loss=1.0264, train=67.10%, val=65.28%
task2_Adam_0.001 06/10: loss=0.9671, train=68.42%, val=65.16%
task2_Adam_0.001 07/10: loss=0.9145, train=69.85%, val=66.32%
task2_Adam_0.001 08/10: loss=0.8678, train=72.13%, val=67.68%
task2_Adam_0.001 09/10: loss=0.8325, train=72.95%, val=68.58%
task2_Adam_0.001 10/10: loss=0.8002, train=74.37%, val=69.92%
task2_Adam_0.01 01/10: loss=2.3574, train=10.06%, val=9.42%
task2_Adam_0.01 02/10: loss=2.3032, train=9.97%, val=10.24%
task2_Adam_0.01 03/10: loss=2.3032, train=9.93%, val=10.64%
task2_Adam_0.01 04/10: loss=2.3032, train=9.99%, val=10.08%
task2_Adam_0.01 05/10: loss=2.3033, train=10.06%, val=9.42%
task2_Adam_0.01 06/10: loss=2.3033, train=10.03%, val=9.76%
task2_Adam_0.01 07/10: loss=2.3035, train=9.99%, val=10.08%
task2_Adam_0.01 08/10: loss=2.3033, train=10.00%, val=10.02%
task2_Adam_0.01 09/10: loss=2.3032, train=10.06%, val=9.42%
task2_Adam_0.01 10/10: loss=2.3034, train=10.03%, val=9.76%
```

Configuration	Clean train (%)	Validation (%)	Gap (pp)	Training loss	Parameters	Minutes
SGD, lr=0.01	49.52	48.08	1.44	1.4083	94538	3.9
SGD, lr=0.05	61.77	59.90	1.87	1.0852	94538	3.9
SGD, lr=0.1	66.56	64.42	2.14	0.9454	94538	3.9
Momentum, lr=0.005	66.14	63.72	2.42	0.9952	94538	3.9
Momentum, lr=0.02	78.23	71.50	6.73	0.6858	94538	3.9
Momentum, lr=0.05	76.95	70.16	6.79	0.7261	94538	4.0
Adam, lr=0.0001	52.75	51.50	1.25	1.3542	94538	3.9
Adam, lr=0.001	74.37	69.92	4.45	0.8002	94538	3.9
Adam, lr=0.01	10.03	9.76	0.27	2.3034	94538	4.0




```
Best rates selected using final validation accuracy: {'SGD': 0.1, 'Momentum': 0.02,
'Adam': 0.001}
```

Interpretation: The best final validation accuracies are **64.42% for SGD (0.1)**, **71.50% for momentum (0.02)**, and **69.92% for Adam (0.001)**. Momentum wins among these settings. Adam at 0.01 stays near chance (**9.76%**) with loss near log(10), showing that a high learning rate can prevent learning. All three optimizers need suitable learning rates.

Increasing the SGD learning rate improves its result throughout the tested range, but this does not mean still larger rates would help. Momentum is best at the middle tested rate, while Adam performs much worse at its highest rate. The best-rate loss curves compare how quickly the chosen runs learn within the same ten-epoch budget. These conclusions apply to this CNN and the tested learning-rate ranges.

Comparison discussion

- Momentum at 0.02 is the best tested CNN setting, reaching 71.50% validation accuracy.
- Adam converges best on the quadratic at its chosen rate, but momentum performs best on the CNN. Results depend on the problem and settings.
- These are single-seed, fixed-budget comparisons; they do not establish that one optimizer is always better.

Task 3: Regularization and residual connections

3(a): Regularization

Seven configurations use a batch-normalized CNN, the same 5,000 training images, and 30 epochs. The baseline is compared with weight decay, dropout, flip/crop, custom cutout, custom mixup, and all combined. Final training accuracy uses clean images.

In [12]:
```
class Cutout:
    """Mask a randomly centred square in each normalized tensor; zero is channel me
    def __init__(self, length=8):
        self.length = length
    def __call__(self, image):
        image = image.clone()
        height, width = image.shape[-2:]
```

```
        y = torch.randint(height, ()).item()
        x = torch.randint(width, ()).item()
        half = self.length // 2
        image[:, max(0, y-half):min(height, y-half+self.length),
              max(0, x-half):min(width, x-half+self.length)] = 0
        return image

def mixup_batch(inputs, labels, alpha=0.2):
    lam = float(np.random.beta(alpha, alpha))
    permutation = torch.randperm(len(inputs), device=inputs.device)
    return lam*inputs + (1-lam)*inputs[permutation], labels, labels[permutation], l

def augmented_dataset(indices, flip_crop=False, cutout=False):
    operations = []
    if flip_crop:
        operations += [transforms.RandomCrop(32, padding=4), transforms.RandomHoriz
    operations += [transforms.ToTensor(), normalize]
    if cutout:
        operations.append(Cutout(8))
    dataset = datasets.CIFAR10(root=DATA_DIR, train=True, download=False,
                               transform=transforms.Compose(operations))
    return Subset(dataset, indices)

regularization_configs = {
    'baseline': {}, 'weight_decay': {'weight_decay': 5e-4},
    'dropout': {'dropout': 0.3}, 'flip_crop': {'flip_crop': True},
    'cutout': {'cutout': True}, 'mixup': {'mixup_alpha': 0.2},
    'all_combined': {'weight_decay': 5e-4, 'dropout': 0.3, 'flip_crop': True,
                     'cutout': True, 'mixup_alpha': 0.2},
}
seed_everything()
preview_data = augmented_dataset(small_train_indices, flip_crop=True, cutout=True)
fig, axes = plt.subplots(1, 5, figsize=(10, 2))
for j, ax in enumerate(axes):
    x, y = preview_data[j]
    ax.imshow(np.clip(x.permute(1, 2, 0).numpy()*std+mean, 0, 1))
    ax.set_title(raw_train.classes[y])
    ax.axis('off')
plt.suptitle('Task 3(a): flip/crop + custom cutout examples')
plt.tight_layout()
plt.show()

task3a_results = {}
for name, settings in regularization_configs.items():
    config = {'architecture': 'SmallCNN_BN', 'optimizer': 'Momentum', 'lr': 0.03,
              'epochs': EPOCHS_T3A, **settings}
    data = augmented_dataset(small_train_indices, settings.get('flip_crop', False),
                             settings.get('cutout', False))
    task3a_results[name] = run_experiment(
        'task3a_'+name,
        lambda settings=settings: SmallCNN(batch_norm=True, dropout=settings.get('d
        config, training_dataset=data, clean_dataset=small_train_dataset,
        mixup_alpha=settings.get('mixup_alpha', 0.0))
comparison_table(task3a_results)
for names in [('baseline', 'weight_decay', 'dropout', 'flip_crop'),
```

```
                ('baseline', 'cutout', 'mixup', 'all_combined')]:
    plot_comparison({name: task3a_results[name] for name in names}, 'Task 3(a): reg
```

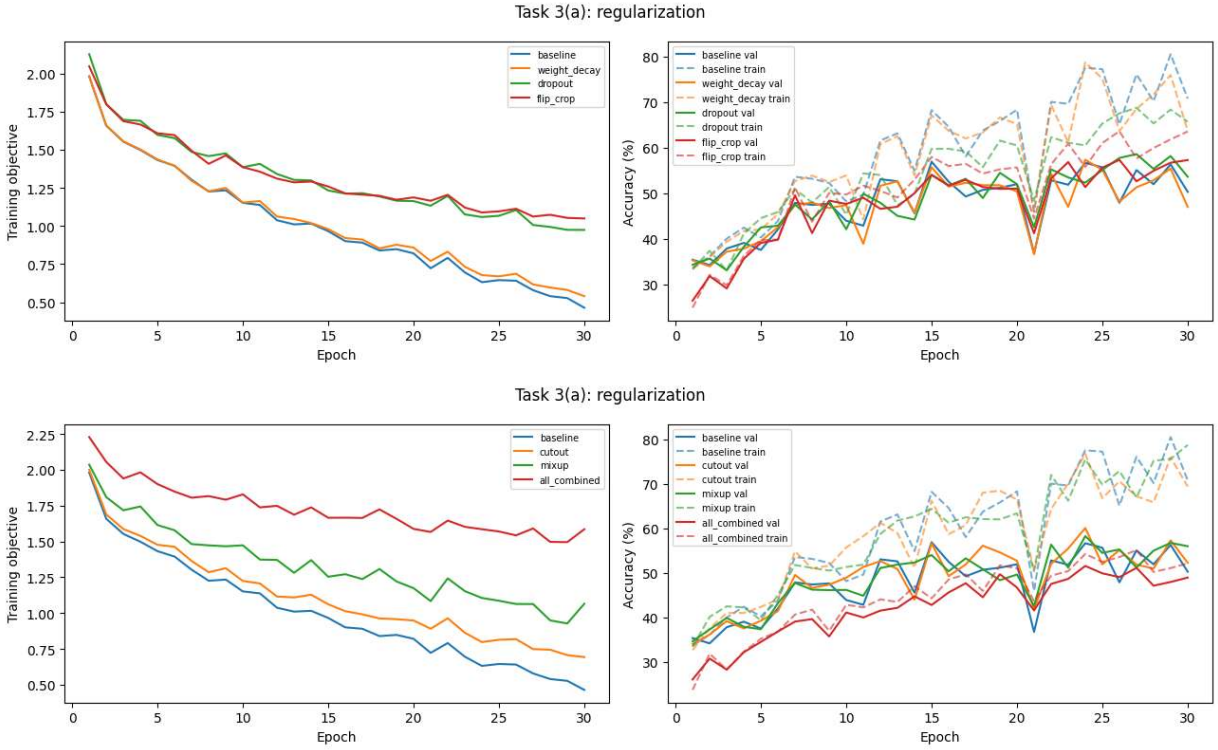
```
task3a_baseline 01/30: loss=1.9806, train=33.64%, val=35.38%
task3a_baseline 02/30: loss=1.6575, train=36.12%, val=34.20%
task3a_baseline 03/30: loss=1.5525, train=40.00%, val=37.82%
task3a_baseline 04/30: loss=1.4977, train=42.42%, val=39.06%
task3a_baseline 05/30: loss=1.4321, train=40.24%, val=37.52%
task3a_baseline 06/30: loss=1.3943, train=43.86%, val=42.02%
task3a_baseline 07/30: loss=1.3026, train=53.56%, val=47.86%
task3a_baseline 08/30: loss=1.2255, train=53.14%, val=47.40%
task3a_baseline 09/30: loss=1.2336, train=52.20%, val=47.64%
task3a_baseline 10/30: loss=1.1511, train=48.16%, val=43.92%
task3a_baseline 11/30: loss=1.1373, train=49.62%, val=42.84%
task3a_baseline 12/30: loss=1.0372, train=61.52%, val=53.04%
task3a_baseline 13/30: loss=1.0099, train=63.10%, val=52.58%
task3a_baseline 14/30: loss=1.0156, train=54.88%, val=45.56%
task3a_baseline 15/30: loss=0.9657, train=68.18%, val=56.88%
task3a_baseline 16/30: loss=0.9001, train=64.54%, val=52.48%
task3a_baseline 17/30: loss=0.8903, train=58.04%, val=49.24%
task3a_baseline 18/30: loss=0.8392, train=63.72%, val=50.70%
task3a_baseline 19/30: loss=0.8476, train=65.80%, val=51.16%
task3a_baseline 20/30: loss=0.8201, train=68.28%, val=51.94%
task3a_baseline 21/30: loss=0.7223, train=45.96%, val=36.76%
task3a_baseline 22/30: loss=0.7906, train=70.00%, val=52.80%
task3a_baseline 23/30: loss=0.6955, train=69.58%, val=51.82%
task3a_baseline 24/30: loss=0.6312, train=77.50%, val=56.64%
task3a_baseline 25/30: loss=0.6446, train=77.18%, val=55.64%
task3a_baseline 26/30: loss=0.6406, train=65.08%, val=47.88%
task3a_baseline 27/30: loss=0.5786, train=76.04%, val=55.04%
task3a_baseline 28/30: loss=0.5396, train=70.16%, val=51.92%
task3a_baseline 29/30: loss=0.5269, train=80.46%, val=56.28%
task3a_baseline 30/30: loss=0.4636, train=70.72%, val=50.26%
task3a_weight_decay 01/30: loss=1.9805, train=33.40%, val=35.24%
task3a_weight_decay 02/30: loss=1.6589, train=35.92%, val=33.90%
task3a_weight_decay 03/30: loss=1.5544, train=39.34%, val=37.14%
task3a_weight_decay 04/30: loss=1.5009, train=41.68%, val=37.80%
task3a_weight_decay 05/30: loss=1.4358, train=42.02%, val=39.44%
task3a_weight_decay 06/30: loss=1.3942, train=45.38%, val=42.52%
task3a_weight_decay 07/30: loss=1.2956, train=52.64%, val=47.38%
task3a_weight_decay 08/30: loss=1.2270, train=53.82%, val=47.92%
task3a_weight_decay 09/30: loss=1.2492, train=52.42%, val=46.72%
task3a_weight_decay 10/30: loss=1.1536, train=53.86%, val=47.34%
task3a_weight_decay 11/30: loss=1.1630, train=44.18%, val=38.86%
task3a_weight_decay 12/30: loss=1.0615, train=60.72%, val=51.60%
task3a_weight_decay 13/30: loss=1.0452, train=62.46%, val=52.56%
task3a_weight_decay 14/30: loss=1.0188, train=53.86%, val=45.70%
task3a_weight_decay 15/30: loss=0.9789, train=67.04%, val=55.64%
task3a_weight_decay 16/30: loss=0.9203, train=63.66%, val=51.42%
task3a_weight_decay 17/30: loss=0.9106, train=61.98%, val=52.24%
task3a_weight_decay 18/30: loss=0.8528, train=63.28%, val=51.72%
task3a_weight_decay 19/30: loss=0.8765, train=66.58%, val=51.72%
task3a_weight_decay 20/30: loss=0.8579, train=65.14%, val=50.34%
task3a_weight_decay 21/30: loss=0.7703, train=45.78%, val=36.58%
task3a_weight_decay 22/30: loss=0.8308, train=69.34%, val=54.24%
task3a_weight_decay 23/30: loss=0.7322, train=61.14%, val=46.98%
task3a_weight_decay 24/30: loss=0.6775, train=78.68%, val=57.32%
task3a_weight_decay 25/30: loss=0.6691, train=75.10%, val=55.08%
task3a_weight_decay 26/30: loss=0.6868, train=63.60%, val=48.12%
```

```
task3a_weight_decay 27/30: loss=0.6162, train=68.46%, val=51.26%
task3a_weight_decay 28/30: loss=0.5959, train=71.64%, val=52.80%
task3a_weight_decay 29/30: loss=0.5802, train=75.88%, val=55.34%
task3a_weight_decay 30/30: loss=0.5394, train=63.66%, val=46.96%
task3a_dropout 01/30: loss=2.1258, train=33.26%, val=34.28%
task3a_dropout 02/30: loss=1.7983, train=37.34%, val=35.62%
task3a_dropout 03/30: loss=1.6957, train=32.98%, val=33.06%
task3a_dropout 04/30: loss=1.6894, train=41.10%, val=38.20%
task3a_dropout 05/30: loss=1.5966, train=44.50%, val=42.46%
task3a_dropout 06/30: loss=1.5758, train=45.84%, val=42.84%
task3a_dropout 07/30: loss=1.4849, train=50.88%, val=47.48%
task3a_dropout 08/30: loss=1.4575, train=47.92%, val=44.22%
task3a_dropout 09/30: loss=1.4755, train=51.32%, val=48.18%
task3a_dropout 10/30: loss=1.3852, train=45.26%, val=42.06%
task3a_dropout 11/30: loss=1.4068, train=54.30%, val=49.86%
task3a_dropout 12/30: loss=1.3403, train=53.96%, val=47.86%
task3a_dropout 13/30: loss=1.3021, train=47.34%, val=45.00%
task3a_dropout 14/30: loss=1.2987, train=49.84%, val=44.18%
task3a_dropout 15/30: loss=1.2324, train=59.70%, val=54.04%
task3a_dropout 16/30: loss=1.2119, train=59.70%, val=51.64%
task3a_dropout 17/30: loss=1.2143, train=59.14%, val=53.14%
task3a_dropout 18/30: loss=1.1932, train=55.66%, val=48.90%
task3a_dropout 19/30: loss=1.1651, train=61.50%, val=54.44%
task3a_dropout 20/30: loss=1.1626, train=60.44%, val=51.94%
task3a_dropout 21/30: loss=1.1324, train=48.36%, val=42.50%
task3a_dropout 22/30: loss=1.1971, train=62.24%, val=55.20%
task3a_dropout 23/30: loss=1.0757, train=61.06%, val=53.42%
task3a_dropout 24/30: loss=1.0582, train=60.44%, val=52.28%
task3a_dropout 25/30: loss=1.0665, train=65.26%, val=55.06%
task3a_dropout 26/30: loss=1.1042, train=67.48%, val=57.70%
task3a_dropout 27/30: loss=1.0059, train=68.70%, val=58.54%
task3a_dropout 28/30: loss=0.9934, train=65.28%, val=55.30%
task3a_dropout 29/30: loss=0.9742, train=68.32%, val=58.12%
task3a_dropout 30/30: loss=0.9738, train=65.64%, val=53.58%
task3a_flip_crop 01/30: loss=2.0464, train=24.84%, val=26.38%
task3a_flip_crop 02/30: loss=1.7974, train=32.06%, val=31.78%
task3a_flip_crop 03/30: loss=1.6867, train=29.78%, val=29.08%
task3a_flip_crop 04/30: loss=1.6651, train=36.22%, val=35.56%
task3a_flip_crop 05/30: loss=1.6074, train=39.66%, val=39.06%
task3a_flip_crop 06/30: loss=1.5953, train=39.80%, val=39.78%
task3a_flip_crop 07/30: loss=1.4928, train=51.18%, val=49.52%
task3a_flip_crop 08/30: loss=1.4072, train=43.70%, val=41.22%
task3a_flip_crop 09/30: loss=1.4621, train=50.02%, val=48.34%
task3a_flip_crop 10/30: loss=1.3850, train=49.66%, val=47.60%
task3a_flip_crop 11/30: loss=1.3556, train=51.62%, val=48.98%
task3a_flip_crop 12/30: loss=1.3101, train=50.48%, val=46.54%
task3a_flip_crop 13/30: loss=1.2856, train=49.02%, val=47.00%
task3a_flip_crop 14/30: loss=1.2911, train=52.96%, val=49.98%
task3a_flip_crop 15/30: loss=1.2586, train=57.92%, val=53.96%
task3a_flip_crop 16/30: loss=1.2132, train=55.92%, val=51.64%
task3a_flip_crop 17/30: loss=1.2049, train=56.40%, val=52.90%
task3a_flip_crop 18/30: loss=1.1975, train=54.26%, val=51.32%
task3a_flip_crop 19/30: loss=1.1716, train=55.20%, val=50.92%
task3a_flip_crop 20/30: loss=1.1869, train=55.62%, val=50.98%
task3a_flip_crop 21/30: loss=1.1649, train=44.38%, val=41.18%
task3a_flip_crop 22/30: loss=1.2039, train=56.30%, val=53.20%
```

```
task3a_flip_crop 23/30: loss=1.1206, train=60.82%, val=56.80%
task3a_flip_crop 24/30: loss=1.0882, train=55.72%, val=51.30%
task3a_flip_crop 25/30: loss=1.0956, train=60.98%, val=55.58%
task3a_flip_crop 26/30: loss=1.1125, train=63.56%, val=57.28%
task3a_flip_crop 27/30: loss=1.0614, train=57.58%, val=52.58%
task3a_flip_crop 28/30: loss=1.0731, train=59.88%, val=54.88%
task3a_flip_crop 29/30: loss=1.0527, train=61.78%, val=56.66%
task3a_flip_crop 30/30: loss=1.0490, train=63.52%, val=57.24%
task3a_cutout 01/30: loss=2.0001, train=32.68%, val=34.04%
task3a_cutout 02/30: loss=1.6881, train=37.60%, val=36.16%
task3a_cutout 03/30: loss=1.5865, train=41.00%, val=39.18%
task3a_cutout 04/30: loss=1.5386, train=41.02%, val=37.54%
task3a_cutout 05/30: loss=1.4763, train=42.38%, val=39.28%
task3a_cutout 06/30: loss=1.4626, train=44.06%, val=41.42%
task3a_cutout 07/30: loss=1.3634, train=54.82%, val=49.54%
task3a_cutout 08/30: loss=1.2847, train=51.00%, val=46.64%
task3a_cutout 09/30: loss=1.3131, train=51.62%, val=47.36%
task3a_cutout 10/30: loss=1.2240, train=55.70%, val=48.92%
task3a_cutout 11/30: loss=1.2064, train=58.24%, val=51.32%
task3a_cutout 12/30: loss=1.1152, train=61.22%, val=52.64%
task3a_cutout 13/30: loss=1.1090, train=58.92%, val=50.86%
task3a_cutout 14/30: loss=1.1281, train=51.16%, val=44.00%
task3a_cutout 15/30: loss=1.0609, train=66.18%, val=56.62%
task3a_cutout 16/30: loss=1.0128, train=58.68%, val=49.28%
task3a_cutout 17/30: loss=0.9911, train=60.72%, val=51.90%
task3a_cutout 18/30: loss=0.9624, train=67.98%, val=56.10%
task3a_cutout 19/30: loss=0.9572, train=68.44%, val=54.60%
task3a_cutout 20/30: loss=0.9482, train=66.42%, val=52.72%
task3a_cutout 21/30: loss=0.8905, train=50.62%, val=41.88%
task3a_cutout 22/30: loss=0.9637, train=64.32%, val=52.02%
task3a_cutout 23/30: loss=0.8628, train=70.04%, val=55.46%
task3a_cutout 24/30: loss=0.7974, train=76.72%, val=60.06%
task3a_cutout 25/30: loss=0.8135, train=66.76%, val=51.92%
task3a_cutout 26/30: loss=0.8182, train=70.54%, val=55.12%
task3a_cutout 27/30: loss=0.7483, train=67.16%, val=51.86%
task3a_cutout 28/30: loss=0.7440, train=65.84%, val=50.96%
task3a_cutout 29/30: loss=0.7062, train=75.92%, val=57.30%
task3a_cutout 30/30: loss=0.6928, train=69.36%, val=52.26%
task3a_mixup 01/30: loss=2.0358, train=33.42%, val=34.66%
task3a_mixup 02/30: loss=1.8090, train=40.14%, val=37.32%
task3a_mixup 03/30: loss=1.7173, train=42.50%, val=39.90%
task3a_mixup 04/30: loss=1.7438, train=42.24%, val=37.92%
task3a_mixup 05/30: loss=1.6142, train=39.38%, val=37.36%
task3a_mixup 06/30: loss=1.5778, train=45.06%, val=43.22%
task3a_mixup 07/30: loss=1.4813, train=51.76%, val=47.74%
task3a_mixup 08/30: loss=1.4725, train=51.10%, val=46.24%
task3a_mixup 09/30: loss=1.4662, train=50.52%, val=46.14%
task3a_mixup 10/30: loss=1.4728, train=51.32%, val=46.18%
task3a_mixup 11/30: loss=1.3731, train=51.84%, val=44.86%
task3a_mixup 12/30: loss=1.3709, train=58.74%, val=51.08%
task3a_mixup 13/30: loss=1.2808, train=61.74%, val=51.86%
task3a_mixup 14/30: loss=1.3694, train=62.66%, val=52.32%
task3a_mixup 15/30: loss=1.2533, train=64.34%, val=54.00%
task3a_mixup 16/30: loss=1.2711, train=61.26%, val=50.36%
task3a_mixup 17/30: loss=1.2369, train=62.48%, val=53.26%
task3a_mixup 18/30: loss=1.3075, train=62.06%, val=50.84%
```

```
task3a_mixup 19/30: loss=1.2205, train=62.00%, val=48.36%
task3a_mixup 20/30: loss=1.1738, train=63.36%, val=49.64%
task3a_mixup 21/30: loss=1.0833, train=50.38%, val=42.56%
task3a_mixup 22/30: loss=1.2419, train=71.98%, val=56.34%
task3a_mixup 23/30: loss=1.1531, train=66.18%, val=51.18%
task3a_mixup 24/30: loss=1.1054, train=75.46%, val=58.26%
task3a_mixup 25/30: loss=1.0859, train=69.76%, val=54.48%
task3a_mixup 26/30: loss=1.0633, train=72.82%, val=55.26%
task3a_mixup 27/30: loss=1.0628, train=66.92%, val=51.20%
task3a_mixup 28/30: loss=0.9488, train=75.12%, val=54.98%
task3a_mixup 29/30: loss=0.9273, train=75.56%, val=56.72%
task3a_mixup 30/30: loss=1.0648, train=78.68%, val=55.98%
task3a_all_combined 01/30: loss=2.2285, train=23.76%, val=26.10%
task3a_all_combined 02/30: loss=2.0546, train=31.88%, val=30.78%
task3a_all_combined 03/30: loss=1.9391, train=28.32%, val=28.30%
task3a_all_combined 04/30: loss=1.9822, train=32.32%, val=32.20%
task3a_all_combined 05/30: loss=1.9000, train=35.22%, val=34.52%
task3a_all_combined 06/30: loss=1.8479, train=36.82%, val=36.86%
task3a_all_combined 07/30: loss=1.8052, train=40.66%, val=39.08%
task3a_all_combined 08/30: loss=1.8163, train=41.76%, val=39.64%
task3a_all_combined 09/30: loss=1.7910, train=37.00%, val=35.74%
task3a_all_combined 10/30: loss=1.8284, train=42.86%, val=41.12%
task3a_all_combined 11/30: loss=1.7374, train=42.26%, val=40.00%
task3a_all_combined 12/30: loss=1.7484, train=44.06%, val=41.54%
task3a_all_combined 13/30: loss=1.6863, train=43.48%, val=42.16%
task3a_all_combined 14/30: loss=1.7377, train=47.00%, val=44.74%
task3a_all_combined 15/30: loss=1.6650, train=44.28%, val=42.80%
task3a_all_combined 16/30: loss=1.6657, train=48.52%, val=45.62%
task3a_all_combined 17/30: loss=1.6639, train=49.68%, val=47.68%
task3a_all_combined 18/30: loss=1.7237, train=45.94%, val=44.52%
task3a_all_combined 19/30: loss=1.6576, train=51.70%, val=49.70%
task3a_all_combined 20/30: loss=1.5875, train=51.02%, val=46.72%
task3a_all_combined 21/30: loss=1.5660, train=43.62%, val=41.58%
task3a_all_combined 22/30: loss=1.6456, train=49.36%, val=47.52%
task3a_all_combined 23/30: loss=1.6020, train=50.40%, val=48.66%
task3a_all_combined 24/30: loss=1.5852, train=54.24%, val=51.56%
task3a_all_combined 25/30: loss=1.5693, train=52.46%, val=49.90%
task3a_all_combined 26/30: loss=1.5425, train=53.56%, val=49.04%
task3a_all_combined 27/30: loss=1.5904, train=55.10%, val=51.12%
task3a_all_combined 28/30: loss=1.4967, train=50.22%, val=47.12%
task3a_all_combined 29/30: loss=1.4951, train=51.08%, val=47.96%
task3a_all_combined 30/30: loss=1.5850, train=52.14%, val=48.94%
```

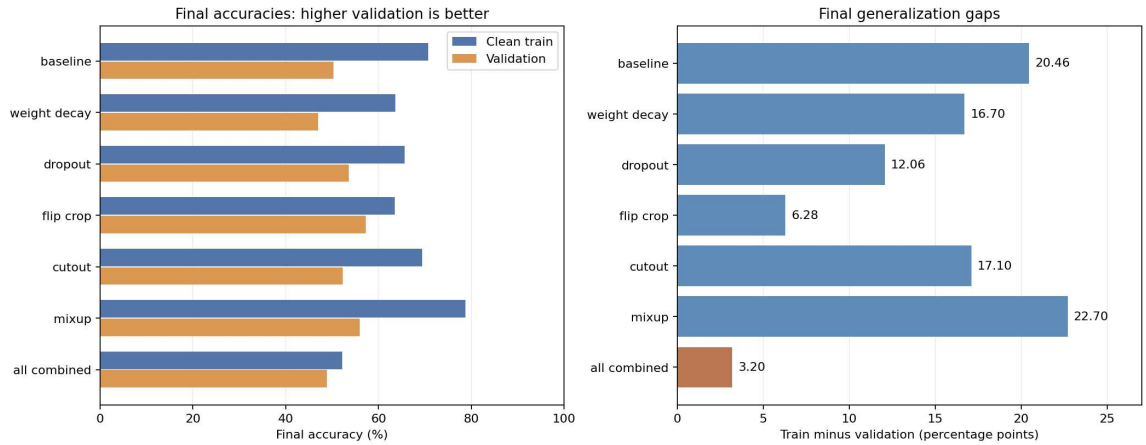
Configuration	Clean train (%)	Validation (%)	Gap (pp)	Training loss	Parameters	Minutes
baseline	70.72	50.26	20.46	0.4636	94762	2.0
weight_decay	63.66	46.96	16.70	0.5394	94762	2.0
dropout	65.64	53.58	12.06	0.9738	94762	2.0
flip_crop	63.52	57.24	6.28	1.0490	94762	2.3
cutout	69.36	52.26	17.10	0.6928	94762	2.1
mixup	78.68	55.98	22.70	1.0648	94762	1.9
all_combined	52.14	48.94	3.20	1.5850	94762	2.4



Interpretation: Flip/crop gives the best validation accuracy (**57.24%**, versus **50.26%** baseline). Dropout, cutout, and mixup also improve validation accuracy; weight decay alone does not in this run. All combined reaches only **48.94%**, suggesting too much regularization or too little training for that combination. Solid accuracy curves show validation; dashed curves show clean training. Mixup's blended-target loss is not directly comparable with ordinary cross-entropy.

Dropout reaches 53.58%, cutout 52.26%, and mixup 55.98% validation accuracy. Random flip/crop changes image position and orientation during training, encouraging features that still work when these details change. The combined model has low training accuracy as well as low validation accuracy, so reducing overfitting alone is not enough to explain its behavior. A different regularization strength or training budget could change the ranking.

Final accuracy and generalization gaps



Interpretation: Gap means clean training accuracy minus validation accuracy. Baseline has a **20.46-point** gap; flip/crop improves validation accuracy and reduces the gap to **6.28 points**. All combined has the smallest gap (**3.20 points**) but low validation accuracy, so a small gap alone is not enough. Mixup improves validation accuracy despite a **22.70-point** gap. This chart uses the saved table values; no training was repeated.

A large positive gap means the model handles its training images better than images it did not train on. Flip/crop gives a useful balance here because validation accuracy increases while the gap becomes smaller. The combined model instead has low accuracy on both sets, which is consistent with underfitting under this training budget. Gap size should always be read alongside the actual validation score.

Comparison discussion

- Flip/crop improves validation accuracy by **6.98 percentage points** over baseline and reduces the train-validation gap.
- Combining every regularizer does not give the best result. Regularization strength must suit the model and training budget.
- Final accuracy can fluctuate, particularly with batch normalization and a small training subset. Repeated seeds would give a more reliable estimate, but were not run here.

3(b): Plain versus residual networks

Depths 8, 20, and 56 follow `6n + 2`, with stage widths 16/32/64, batch normalization, Kaiming initialization, and global average pooling. Each plain/residual pair differs only by its shortcut. Shape changes use spatial subsampling and zero-padded channels, adding no parameters. All six models use the same 30-epoch protocol.

In [13]:
```
class CifarBlock(nn.Module):
    def __init__(self, in_channels, out_channels, stride=1, residual=True):
        super().__init__()
```

```
        self.conv1 = nn.Conv2d(in_channels, out_channels, 3, stride, 1, bias=False)
        self.bn1 = nn.BatchNorm2d(out_channels)
        self.conv2 = nn.Conv2d(out_channels, out_channels, 3, padding=1, bias=False
        self.bn2 = nn.BatchNorm2d(out_channels)
        self.residual = residual
        self.stride = stride
        self.out_channels = out_channels
    def forward(self, x):
        out = torch.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        if self.residual:
            shortcut = x[:, :, ::self.stride, ::self.stride]
            difference = self.out_channels - shortcut.shape[1]
            if difference:
                left = difference // 2
                shortcut = nn.functional.pad(shortcut, (0, 0, 0, 0, left, differenc
            out = out + shortcut
        return torch.relu(out)

class CifarNetwork(nn.Module):
    def __init__(self, depth=20, residual=True, widths=(16, 32, 64)):
        super().__init__()
        if depth < 8 or (depth-2) % 6:
            raise ValueError('Depth must equal 6n+2 with n >= 1')
        blocks_per_stage = (depth-2)//6
        self.stem = nn.Sequential(nn.Conv2d(3, widths[0], 3, padding=1, bias=False)
                                  nn.BatchNorm2d(widths[0]), nn.ReLU())
        blocks = []
        channels = widths[0]
        for stage, width in enumerate(widths):
            for block in range(blocks_per_stage):
                stride = 2 if stage > 0 and block == 0 else 1
                blocks.append(CifarBlock(channels, width, stride, residual))
                channels = width
        self.blocks = nn.Sequential(*blocks)
        self.head = nn.Sequential(nn.AdaptiveAvgPool2d(1), nn.Flatten(), nn.Linear(
        initialize_network(self)
    def forward(self, x):
        return self.head(self.blocks(self.stem(x)))

for depth in (8, 20, 56):
    plain = CifarNetwork(depth, False)
    residual = CifarNetwork(depth, True)
    assert sum(isinstance(m, (nn.Conv2d, nn.Linear)) for m in plain.modules()) == d
    assert sum(p.numel() for p in plain.parameters()) == sum(p.numel() for p in res
    print(f'Depth {depth}: {sum(p.numel() for p in plain.parameters()):,} parameter
del plain, residual

task3b_results = {}
depth_train_dataset = augmented_dataset(train_indices, flip_crop=True)
for depth in (8, 20, 56):
    for residual in (False, True):
        name = f'{"residual" if residual else "plain"}_{depth}'
        config = {'architecture': 'CifarNetwork', 'depth': depth, 'residual': resid
                  'widths': [16, 32, 64], 'optimizer': 'Momentum', 'lr': 0.05,
                  'weight_decay': 5e-4, 'cosine': True, 'flip_crop': True,
```

```
                  'epochs': EPOCHS_T3B}
        task3b_results[name] = run_experiment(
            'task3b_'+name, lambda d=depth, r=residual: CifarNetwork(d, r), config,
            training_dataset=depth_train_dataset)
comparison_table(task3b_results)
fig, axes = plt.subplots(1, 3, figsize=(15, 4), sharey=True)
for ax, depth in zip(axes, (8, 20, 56)):
    for family in ('plain', 'residual'):
        history = task3b_results[f'{family}_{depth}']['history']
        ax.plot([r['epoch'] for r in history], [r['train_loss'] for r in history],
    ax.set(title=f'Depth {depth}', xlabel='Epoch', ylabel='Training loss')
    ax.legend()
plt.suptitle('Task 3(b): shortcut is the only architectural difference within each
plt.tight_layout()
plt.show()
```

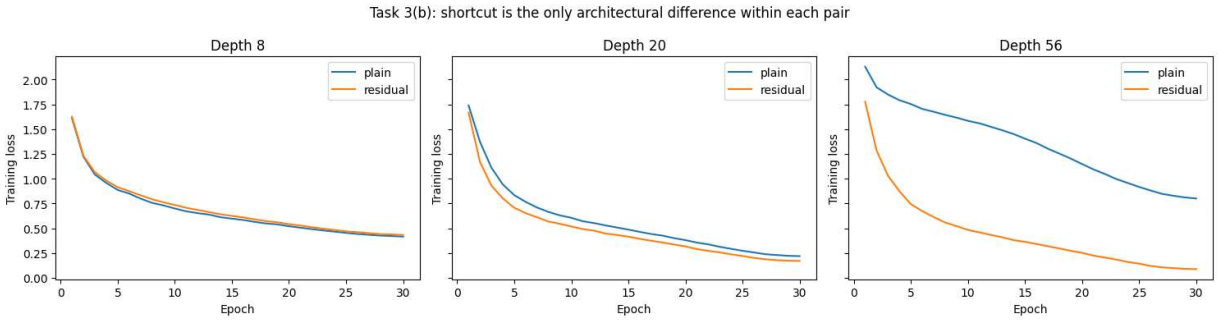
```
Depth 8: 75,290 parameters in each family
Depth 20: 269,722 parameters in each family
Depth 56: 853,018 parameters in each family
task3b_plain_8 01/30: loss=1.6103, train=47.19%, val=47.48%
task3b_plain_8 02/30: loss=1.2220, train=52.71%, val=53.24%
task3b_plain_8 03/30: loss=1.0419, train=63.38%, val=62.50%
task3b_plain_8 04/30: loss=0.9573, train=67.27%, val=66.24%
task3b_plain_8 05/30: loss=0.8864, train=68.97%, val=67.72%
task3b_plain_8 06/30: loss=0.8500, train=62.60%, val=61.78%
task3b_plain_8 07/30: loss=0.7990, train=68.22%, val=66.58%
task3b_plain_8 08/30: loss=0.7565, train=68.88%, val=66.94%
task3b_plain_8 09/30: loss=0.7306, train=68.18%, val=66.16%
task3b_plain_8 10/30: loss=0.6993, train=66.26%, val=66.08%
task3b_plain_8 11/30: loss=0.6707, train=75.76%, val=74.10%
task3b_plain_8 12/30: loss=0.6523, train=74.96%, val=73.34%
task3b_plain_8 13/30: loss=0.6378, train=74.21%, val=72.64%
task3b_plain_8 14/30: loss=0.6111, train=76.07%, val=74.38%
task3b_plain_8 15/30: loss=0.5966, train=73.43%, val=72.38%
task3b_plain_8 16/30: loss=0.5831, train=79.24%, val=76.48%
task3b_plain_8 17/30: loss=0.5640, train=78.17%, val=76.04%
task3b_plain_8 18/30: loss=0.5479, train=80.89%, val=78.52%
task3b_plain_8 19/30: loss=0.5386, train=80.85%, val=79.06%
task3b_plain_8 20/30: loss=0.5196, train=80.81%, val=79.38%
task3b_plain_8 21/30: loss=0.5057, train=82.99%, val=80.44%
task3b_plain_8 22/30: loss=0.4910, train=82.59%, val=80.14%
task3b_plain_8 23/30: loss=0.4774, train=83.20%, val=81.94%
task3b_plain_8 24/30: loss=0.4657, train=83.25%, val=80.72%
task3b_plain_8 25/30: loss=0.4528, train=85.31%, val=82.96%
task3b_plain_8 26/30: loss=0.4415, train=85.72%, val=83.18%
task3b_plain_8 27/30: loss=0.4337, train=86.28%, val=83.48%
task3b_plain_8 28/30: loss=0.4253, train=86.42%, val=83.62%
task3b_plain_8 29/30: loss=0.4212, train=86.58%, val=84.04%
task3b_plain_8 30/30: loss=0.4156, train=86.68%, val=83.90%
task3b_residual_8 01/30: loss=1.6253, train=39.47%, val=39.76%
task3b_residual_8 02/30: loss=1.2320, train=50.21%, val=50.88%
task3b_residual_8 03/30: loss=1.0646, train=52.73%, val=51.64%
task3b_residual_8 04/30: loss=0.9813, train=64.03%, val=63.12%
task3b_residual_8 05/30: loss=0.9145, train=56.38%, val=54.84%
task3b_residual_8 06/30: loss=0.8740, train=60.78%, val=60.42%
task3b_residual_8 07/30: loss=0.8328, train=67.18%, val=66.16%
task3b_residual_8 08/30: loss=0.7925, train=63.30%, val=62.18%
task3b_residual_8 09/30: loss=0.7622, train=71.68%, val=70.10%
task3b_residual_8 10/30: loss=0.7345, train=69.23%, val=67.54%
task3b_residual_8 11/30: loss=0.7062, train=70.74%, val=69.48%
task3b_residual_8 12/30: loss=0.6844, train=74.45%, val=73.20%
task3b_residual_8 13/30: loss=0.6629, train=74.70%, val=73.72%
task3b_residual_8 14/30: loss=0.6407, train=71.41%, val=70.20%
task3b_residual_8 15/30: loss=0.6248, train=72.17%, val=70.84%
task3b_residual_8 16/30: loss=0.6093, train=79.75%, val=77.50%
task3b_residual_8 17/30: loss=0.5894, train=77.77%, val=75.84%
task3b_residual_8 18/30: loss=0.5712, train=73.77%, val=71.84%
task3b_residual_8 19/30: loss=0.5601, train=78.40%, val=76.06%
task3b_residual_8 20/30: loss=0.5402, train=78.08%, val=76.74%
task3b_residual_8 21/30: loss=0.5275, train=82.47%, val=80.16%
task3b_residual_8 22/30: loss=0.5100, train=81.58%, val=78.78%
task3b_residual_8 23/30: loss=0.4961, train=82.73%, val=81.30%
```

```
task3b_residual_8 24/30: loss=0.4823, train=83.73%, val=82.02%
task3b_residual_8 25/30: loss=0.4684, train=84.77%, val=82.64%
task3b_residual_8 26/30: loss=0.4600, train=85.17%, val=82.58%
task3b_residual_8 27/30: loss=0.4505, train=85.92%, val=82.80%
task3b_residual_8 28/30: loss=0.4401, train=86.02%, val=83.26%
task3b_residual_8 29/30: loss=0.4370, train=86.06%, val=83.46%
task3b_residual_8 30/30: loss=0.4318, train=86.14%, val=83.60%
task3b_plain_20 01/30: loss=1.7391, train=37.81%, val=38.38%
task3b_plain_20 02/30: loss=1.3715, train=44.57%, val=44.36%
task3b_plain_20 03/30: loss=1.1110, train=57.14%, val=56.94%
task3b_plain_20 04/30: loss=0.9419, train=60.68%, val=60.06%
task3b_plain_20 05/30: loss=0.8340, train=72.24%, val=70.20%
task3b_plain_20 06/30: loss=0.7657, train=72.99%, val=70.60%
task3b_plain_20 07/30: loss=0.7088, train=72.13%, val=70.36%
task3b_plain_20 08/30: loss=0.6645, train=77.58%, val=75.70%
task3b_plain_20 09/30: loss=0.6295, train=74.47%, val=73.14%
task3b_plain_20 10/30: loss=0.6051, train=75.23%, val=74.50%
task3b_plain_20 11/30: loss=0.5706, train=72.91%, val=70.66%
task3b_plain_20 12/30: loss=0.5518, train=79.82%, val=77.98%
task3b_plain_20 13/30: loss=0.5283, train=77.27%, val=76.02%
task3b_plain_20 14/30: loss=0.5073, train=81.58%, val=79.64%
task3b_plain_20 15/30: loss=0.4859, train=80.98%, val=78.42%
task3b_plain_20 16/30: loss=0.4631, train=81.10%, val=79.00%
task3b_plain_20 17/30: loss=0.4416, train=85.82%, val=82.90%
task3b_plain_20 18/30: loss=0.4250, train=86.04%, val=82.98%
task3b_plain_20 19/30: loss=0.3996, train=83.83%, val=81.22%
task3b_plain_20 20/30: loss=0.3797, train=85.63%, val=82.98%
task3b_plain_20 21/30: loss=0.3548, train=87.39%, val=83.36%
task3b_plain_20 22/30: loss=0.3371, train=87.99%, val=84.24%
task3b_plain_20 23/30: loss=0.3125, train=90.02%, val=85.82%
task3b_plain_20 24/30: loss=0.2917, train=91.23%, val=86.76%
task3b_plain_20 25/30: loss=0.2721, train=92.00%, val=87.08%
task3b_plain_20 26/30: loss=0.2556, train=92.51%, val=87.72%
task3b_plain_20 27/30: loss=0.2375, train=93.41%, val=88.42%
task3b_plain_20 28/30: loss=0.2294, train=93.80%, val=88.36%
task3b_plain_20 29/30: loss=0.2215, train=93.92%, val=88.62%
task3b_plain_20 30/30: loss=0.2186, train=93.94%, val=88.50%
task3b_residual_20 01/30: loss=1.6682, train=46.24%, val=45.66%
task3b_residual_20 02/30: loss=1.1691, train=64.27%, val=63.40%
task3b_residual_20 03/30: loss=0.9296, train=65.15%, val=64.16%
task3b_residual_20 04/30: loss=0.8005, train=72.65%, val=71.94%
task3b_residual_20 05/30: loss=0.7069, train=76.34%, val=74.62%
task3b_residual_20 06/30: loss=0.6520, train=77.99%, val=76.66%
task3b_residual_20 07/30: loss=0.6110, train=77.66%, val=76.16%
task3b_residual_20 08/30: loss=0.5673, train=76.16%, val=74.18%
task3b_residual_20 09/30: loss=0.5444, train=79.68%, val=78.46%
task3b_residual_20 10/30: loss=0.5182, train=79.74%, val=78.00%
task3b_residual_20 11/30: loss=0.4913, train=76.97%, val=75.24%
task3b_residual_20 12/30: loss=0.4751, train=83.90%, val=82.14%
task3b_residual_20 13/30: loss=0.4460, train=85.03%, val=82.94%
task3b_residual_20 14/30: loss=0.4311, train=81.31%, val=80.00%
task3b_residual_20 15/30: loss=0.4148, train=80.78%, val=78.36%
task3b_residual_20 16/30: loss=0.3928, train=83.92%, val=82.24%
task3b_residual_20 17/30: loss=0.3738, train=85.97%, val=82.82%
task3b_residual_20 18/30: loss=0.3553, train=87.56%, val=84.54%
task3b_residual_20 19/30: loss=0.3355, train=87.19%, val=82.88%
```

```
task3b_residual_20 20/30: loss=0.3161, train=89.50%, val=85.98%
task3b_residual_20 21/30: loss=0.2893, train=91.38%, val=87.42%
task3b_residual_20 22/30: loss=0.2715, train=91.27%, val=87.10%
task3b_residual_20 23/30: loss=0.2556, train=91.41%, val=87.12%
task3b_residual_20 24/30: loss=0.2363, train=93.12%, val=88.22%
task3b_residual_20 25/30: loss=0.2193, train=93.66%, val=88.66%
task3b_residual_20 26/30: loss=0.2002, train=94.32%, val=89.16%
task3b_residual_20 27/30: loss=0.1869, train=94.86%, val=89.72%
task3b_residual_20 28/30: loss=0.1767, train=95.07%, val=89.66%
task3b_residual_20 29/30: loss=0.1724, train=95.28%, val=89.66%
task3b_residual_20 30/30: loss=0.1697, train=95.33%, val=89.68%
task3b_plain_56 01/30: loss=2.1309, train=22.26%, val=21.90%
task3b_plain_56 02/30: loss=1.9228, train=28.37%, val=29.20%
task3b_plain_56 03/30: loss=1.8495, train=30.68%, val=30.70%
task3b_plain_56 04/30: loss=1.7927, train=29.94%, val=29.96%
task3b_plain_56 05/30: loss=1.7537, train=31.33%, val=31.28%
task3b_plain_56 06/30: loss=1.7044, train=34.23%, val=33.82%
task3b_plain_56 07/30: loss=1.6759, train=39.14%, val=38.80%
task3b_plain_56 08/30: loss=1.6446, train=39.33%, val=39.04%
task3b_plain_56 09/30: loss=1.6165, train=40.04%, val=38.80%
task3b_plain_56 10/30: loss=1.5842, train=37.36%, val=36.84%
task3b_plain_56 11/30: loss=1.5593, train=44.02%, val=43.42%
task3b_plain_56 12/30: loss=1.5245, train=43.47%, val=42.78%
task3b_plain_56 13/30: loss=1.4893, train=45.37%, val=45.10%
task3b_plain_56 14/30: loss=1.4508, train=47.65%, val=46.66%
task3b_plain_56 15/30: loss=1.4043, train=48.39%, val=48.54%
task3b_plain_56 16/30: loss=1.3605, train=50.05%, val=49.66%
task3b_plain_56 17/30: loss=1.3032, train=47.74%, val=47.08%
task3b_plain_56 18/30: loss=1.2538, train=52.57%, val=52.22%
task3b_plain_56 19/30: loss=1.2034, train=56.79%, val=55.98%
task3b_plain_56 20/30: loss=1.1478, train=55.74%, val=55.34%
task3b_plain_56 21/30: loss=1.0938, train=56.51%, val=55.14%
task3b_plain_56 22/30: loss=1.0476, train=61.60%, val=60.24%
task3b_plain_56 23/30: loss=0.9966, train=66.00%, val=63.92%
task3b_plain_56 24/30: loss=0.9567, train=64.73%, val=62.50%
task3b_plain_56 25/30: loss=0.9167, train=68.96%, val=66.08%
task3b_plain_56 26/30: loss=0.8808, train=70.66%, val=67.72%
task3b_plain_56 27/30: loss=0.8471, train=71.89%, val=69.42%
task3b_plain_56 28/30: loss=0.8279, train=73.15%, val=70.52%
task3b_plain_56 29/30: loss=0.8111, train=74.06%, val=71.00%
task3b_plain_56 30/30: loss=0.8000, train=74.17%, val=70.86%
task3b_residual_56 01/30: loss=1.7771, train=43.78%, val=43.90%
task3b_residual_56 02/30: loss=1.2861, train=58.23%, val=57.92%
task3b_residual_56 03/30: loss=1.0299, train=64.75%, val=63.72%
task3b_residual_56 04/30: loss=0.8733, train=70.95%, val=70.02%
task3b_residual_56 05/30: loss=0.7433, train=75.25%, val=73.74%
task3b_residual_56 06/30: loss=0.6729, train=75.61%, val=73.98%
task3b_residual_56 07/30: loss=0.6130, train=79.25%, val=77.20%
task3b_residual_56 08/30: loss=0.5575, train=79.09%, val=76.92%
task3b_residual_56 09/30: loss=0.5215, train=81.54%, val=79.28%
task3b_residual_56 10/30: loss=0.4829, train=81.40%, val=78.90%
task3b_residual_56 11/30: loss=0.4581, train=77.84%, val=75.72%
task3b_residual_56 12/30: loss=0.4327, train=85.15%, val=83.02%
task3b_residual_56 13/30: loss=0.4087, train=82.76%, val=79.66%
task3b_residual_56 14/30: loss=0.3803, train=84.45%, val=81.26%
task3b_residual_56 15/30: loss=0.3623, train=81.31%, val=78.36%
```

```
task3b_residual_56 16/30: loss=0.3409, train=87.73%, val=83.64%
task3b_residual_56 17/30: loss=0.3182, train=88.36%, val=84.56%
task3b_residual_56 18/30: loss=0.2966, train=88.90%, val=84.68%
task3b_residual_56 19/30: loss=0.2721, train=90.01%, val=85.46%
task3b_residual_56 20/30: loss=0.2512, train=91.24%, val=86.38%
task3b_residual_56 21/30: loss=0.2232, train=92.92%, val=87.98%
task3b_residual_56 22/30: loss=0.2041, train=90.73%, val=86.16%
task3b_residual_56 23/30: loss=0.1830, train=94.94%, val=89.14%
task3b_residual_56 24/30: loss=0.1597, train=96.02%, val=89.74%
task3b_residual_56 25/30: loss=0.1419, train=96.56%, val=90.02%
task3b_residual_56 26/30: loss=0.1193, train=97.00%, val=90.30%
task3b_residual_56 27/30: loss=0.1054, train=97.57%, val=90.52%
task3b_residual_56 28/30: loss=0.0971, train=97.88%, val=90.98%
task3b_residual_56 29/30: loss=0.0898, train=98.02%, val=90.82%
task3b_residual_56 30/30: loss=0.0869, train=98.02%, val=90.90%
```

Configuration	Clean train (%)	Validation (%)	Gap (pp)	Training loss	Parameters	Minutes
plain_8	86.68	83.90	2.78	0.4156	75290	18.5
residual_8	86.14	83.60	2.54	0.4318	75290	18.7
plain_20	93.94	88.50	5.44	0.2186	269722	21.3
residual_20	95.33	89.68	5.65	0.1697	269722	21.6
plain_56	74.17	70.86	3.31	0.8000	853018	34.0
residual_56	98.02	90.90	7.12	0.0869	853018	35.2



Interpretation: At depth 8, plain and residual models are similar (**83.90% vs 83.60%** validation). At depth 20, residual improves accuracy to **89.68%**, versus **88.50%**. At depth 56, plain training loss is **0.8000**, compared with **0.0869** for residual, and validation accuracy is **70.86% vs 90.90%**. The worse training loss shows an optimization problem in the deep plain model. Shortcuts help gradients pass through the network.

If the main problem were overfitting, the deeper plain model would normally fit training data well but struggle on validation data. Here it also fits the training data poorly, supporting the optimization explanation. Each pair has the same parameter count because its shortcuts add no learned weights. The results therefore show a much larger shortcut benefit at depth 56 than at depth 8.

Comparison discussion

- At depth 56, shortcuts improve validation accuracy by **20.04 percentage points** and greatly lower training loss.
- Plain/residual pairs have equal parameter counts, so extra learned parameters do not explain the improvement.
- A small train-validation gap is not enough: plain-56 has a smaller gap than residual-56 but much worse accuracy on both sets.

Task 4: Final CNN

The final depth-14 CNN uses stage widths 24/48/96, **386,626 parameters**, and **20 epochs**, meeting both limits. Three ablations remove shortcuts, augmentation, or weight decay. Selection uses validation accuracy; the selected saved model is then evaluated on the test set once.

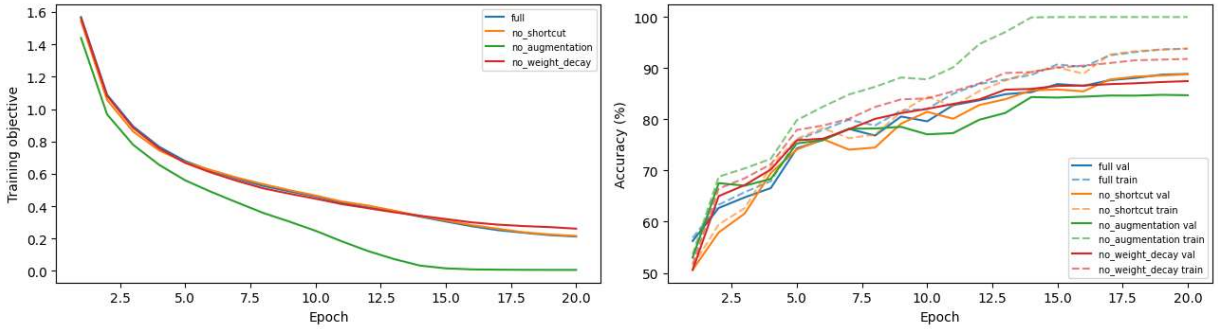
In [14]:
```
assert 1 <= EPOCHS_T4 <= 20
FINAL_WIDTHS = (24, 48, 96)
final_variants = {
    'full': {'residual': True, 'flip_crop': True, 'weight_decay': 5e-4},
    'no_shortcut': {'residual': False, 'flip_crop': True, 'weight_decay': 5e-4},
    'no_augmentation': {'residual': True, 'flip_crop': False, 'weight_decay': 5e-4}
    'no_weight_decay': {'residual': True, 'flip_crop': True, 'weight_decay': 0.0},
}
parameter_count = sum(p.numel() for p in CifarNetwork(14, True, FINAL_WIDTHS).param
assert parameter_count <= 500_000, parameter_count
print(f'Final architecture: depth 14, widths {FINAL_WIDTHS}, {parameter_count:,} pa
task4_results = {}
for name, variant in final_variants.items():
    config = {'architecture': 'CifarNetwork', 'depth': 14, 'widths': list(FINAL_WID
              'optimizer': 'Momentum', 'lr': 0.05, 'cosine': True,
              'epochs': EPOCHS_T4, **variant}
    data = augmented_dataset(train_indices, flip_crop=variant['flip_crop'])
    task4_results[name] = run_experiment(
        'task4_'+name, lambda r=variant['residual']: CifarNetwork(14, r, FINAL_WIDT
        config, training_dataset=data)
comparison_table(task4_results)
plot_comparison(task4_results, 'Task 4: controlled design ablations')
complete_candidates = {name: r for name, r in task4_results.items() if r['status']
if not complete_candidates:
    raise RuntimeError('No completed Task 4 candidates; inspect training settings')
final_name = max(complete_candidates,
                 key=lambda name: complete_candidates[name]['history'][-1]['val_acc
final_run = complete_candidates[final_name]
print('Final choice (highest final validation accuracy):', final_name)
print('Checkpoint:', final_run['checkpoint'])
if task4_results['full']['status'] == 'complete':
    full_accuracy = task4_results['full']['history'][-1]['val_accuracy']
    show_table([[name, f"{100*(r['history'][-1]['val_accuracy']-full_accuracy):+.2f
                 for name, r in complete_candidates.items()],
                ['Variant', 'Validation change vs full (percentage points)']
```

```
Final architecture: depth 14, widths (24, 48, 96), 386,626 parameters
task4_full 01/20: loss=1.5685, train=56.84%, val=56.22%
task4_full 02/20: loss=1.0889, train=63.33%, val=62.68%
task4_full 03/20: loss=0.8933, train=65.78%, val=64.76%
task4_full 04/20: loss=0.7674, train=67.93%, val=66.58%
task4_full 05/20: loss=0.6787, train=75.94%, val=74.34%
task4_full 06/20: loss=0.6183, train=77.94%, val=75.98%
task4_full 07/20: loss=0.5691, train=79.92%, val=78.14%
task4_full 08/20: loss=0.5250, train=78.81%, val=76.84%
task4_full 09/20: loss=0.4916, train=81.75%, val=80.56%
task4_full 10/20: loss=0.4604, train=82.05%, val=79.62%
task4_full 11/20: loss=0.4221, train=85.02%, val=82.78%
task4_full 12/20: loss=0.3926, train=86.90%, val=83.72%
task4_full 13/20: loss=0.3683, train=87.76%, val=84.92%
task4_full 14/20: loss=0.3358, train=88.70%, val=85.32%
task4_full 15/20: loss=0.3055, train=90.74%, val=86.90%
task4_full 16/20: loss=0.2760, train=90.26%, val=86.54%
task4_full 17/20: loss=0.2522, train=92.51%, val=87.66%
task4_full 18/20: loss=0.2359, train=93.17%, val=88.10%
task4_full 19/20: loss=0.2212, train=93.68%, val=88.74%
task4_full 20/20: loss=0.2127, train=93.81%, val=88.88%
task4_no_shortcut 01/20: loss=1.5440, train=51.33%, val=50.60%
task4_no_shortcut 02/20: loss=1.0564, train=59.46%, val=57.90%
task4_no_shortcut 03/20: loss=0.8612, train=62.67%, val=61.60%
task4_no_shortcut 04/20: loss=0.7446, train=70.75%, val=69.52%
task4_no_shortcut 05/20: loss=0.6706, train=76.23%, val=74.10%
task4_no_shortcut 06/20: loss=0.6226, train=78.28%, val=76.14%
task4_no_shortcut 07/20: loss=0.5755, train=76.32%, val=74.08%
task4_no_shortcut 08/20: loss=0.5359, train=77.01%, val=74.50%
task4_no_shortcut 09/20: loss=0.4993, train=81.44%, val=79.16%
task4_no_shortcut 10/20: loss=0.4659, train=84.48%, val=81.48%
task4_no_shortcut 11/20: loss=0.4290, train=82.56%, val=80.12%
task4_no_shortcut 12/20: loss=0.4048, train=85.53%, val=82.80%
task4_no_shortcut 13/20: loss=0.3729, train=87.59%, val=83.94%
task4_no_shortcut 14/20: loss=0.3396, train=89.28%, val=85.68%
task4_no_shortcut 15/20: loss=0.3115, train=90.17%, val=85.84%
task4_no_shortcut 16/20: loss=0.2832, train=88.92%, val=85.48%
task4_no_shortcut 17/20: loss=0.2605, train=92.66%, val=87.78%
task4_no_shortcut 18/20: loss=0.2388, train=93.34%, val=88.36%
task4_no_shortcut 19/20: loss=0.2252, train=93.61%, val=88.58%
task4_no_shortcut 20/20: loss=0.2164, train=93.85%, val=88.80%
task4_no_augmentation 01/20: loss=1.4391, train=53.62%, val=52.96%
task4_no_augmentation 02/20: loss=0.9675, train=68.79%, val=67.54%
task4_no_augmentation 03/20: loss=0.7795, train=70.43%, val=67.04%
task4_no_augmentation 04/20: loss=0.6554, train=72.28%, val=68.36%
task4_no_augmentation 05/20: loss=0.5597, train=79.84%, val=75.30%
task4_no_augmentation 06/20: loss=0.4883, train=82.47%, val=75.96%
task4_no_augmentation 07/20: loss=0.4223, train=84.87%, val=78.16%
task4_no_augmentation 08/20: loss=0.3580, train=86.35%, val=78.22%
task4_no_augmentation 09/20: loss=0.3048, train=88.18%, val=78.54%
task4_no_augmentation 10/20: loss=0.2481, train=87.81%, val=77.08%
task4_no_augmentation 11/20: loss=0.1836, train=90.18%, val=77.32%
task4_no_augmentation 12/20: loss=0.1238, train=94.71%, val=79.92%
task4_no_augmentation 13/20: loss=0.0742, train=97.06%, val=81.26%
task4_no_augmentation 14/20: loss=0.0333, train=99.93%, val=84.36%
task4_no_augmentation 15/20: loss=0.0161, train=100.00%, val=84.26%
```

```
task4_no_augmentation 16/20: loss=0.0097, train=100.00%, val=84.44%
task4_no_augmentation 17/20: loss=0.0079, train=100.00%, val=84.66%
task4_no_augmentation 18/20: loss=0.0070, train=100.00%, val=84.64%
task4_no_augmentation 19/20: loss=0.0066, train=100.00%, val=84.80%
task4_no_augmentation 20/20: loss=0.0065, train=100.00%, val=84.72%
task4_no_weight_decay 01/20: loss=1.5616, train=51.69%, val=50.58%
task4_no_weight_decay 02/20: loss=1.0810, train=66.47%, val=65.00%
task4_no_weight_decay 03/20: loss=0.8843, train=68.49%, val=67.16%
task4_no_weight_decay 04/20: loss=0.7575, train=71.24%, val=70.20%
task4_no_weight_decay 05/20: loss=0.6676, train=77.93%, val=75.90%
task4_no_weight_decay 06/20: loss=0.6074, train=78.79%, val=76.22%
task4_no_weight_decay 07/20: loss=0.5558, train=80.12%, val=78.08%
task4_no_weight_decay 08/20: loss=0.5103, train=82.42%, val=80.10%
task4_no_weight_decay 09/20: loss=0.4770, train=83.91%, val=81.22%
task4_no_weight_decay 10/20: loss=0.4464, train=84.11%, val=82.06%
task4_no_weight_decay 11/20: loss=0.4135, train=85.48%, val=83.02%
task4_no_weight_decay 12/20: loss=0.3880, train=87.02%, val=83.90%
task4_no_weight_decay 13/20: loss=0.3631, train=89.06%, val=85.80%
task4_no_weight_decay 14/20: loss=0.3413, train=89.25%, val=85.96%
task4_no_weight_decay 15/20: loss=0.3206, train=90.14%, val=86.54%
task4_no_weight_decay 16/20: loss=0.3001, train=90.50%, val=86.62%
task4_no_weight_decay 17/20: loss=0.2862, train=91.01%, val=86.88%
task4_no_weight_decay 18/20: loss=0.2772, train=91.56%, val=87.06%
task4_no_weight_decay 19/20: loss=0.2708, train=91.68%, val=87.30%
task4_no_weight_decay 20/20: loss=0.2612, train=91.82%, val=87.48%
```

Configuration	Clean train (%)	Validation (%)	Gap (pp)	Training loss	Parameters	Minutes
full	93.81	88.88	4.93	0.2127	386626	14.0
no_shortcut	93.85	88.80	5.05	0.2164	386626	13.6
no_augmentation	100.00	84.72	15.28	0.0065	386626	12.0
no_weight_decay	91.82	87.48	4.34	0.2612	386626	14.3

Task 4: controlled design ablations




```
Final choice (highest final validation accuracy): full
Checkpoint: /content/drive/MyDrive/EN4554_220381M/outputs/task4_full_a1a89e77b041.pt
```

Variant	Validation change vs full (percentage points)
full	+0.00
no_shortcut	-0.08
no_augmentation	-4.16
no_weight_decay	-1.40

Interpretation: The full model is selected with **88.88% validation accuracy**. Removing augmentation lowers accuracy by **4.16 percentage points** and gives 100% training accuracy, showing overfitting. Removing weight decay lowers validation accuracy by **1.40 points.** Removing shortcuts changes it by only **0.08 points**, too small for a strong claim from one seed. Augmentation has the largest measured benefit.

An ablation changes one design choice while keeping the remaining settings fixed. Without augmentation, perfect training accuracy does not translate into the best validation result, so memorizing the training set is insufficient. The selected full model stays within both the parameter and epoch limits. Selection uses validation results only, keeping the test set separate until the final evaluation.

Ablation discussion

- Changing one choice at a time makes each comparison easier to interpret.
- Augmentation gives the largest measured benefit; weight decay also helps in this final-model setup.
- The shortcut benefit is small at depth 14, unlike the large benefit at depth 56. The full model is selected before inspecting test performance.

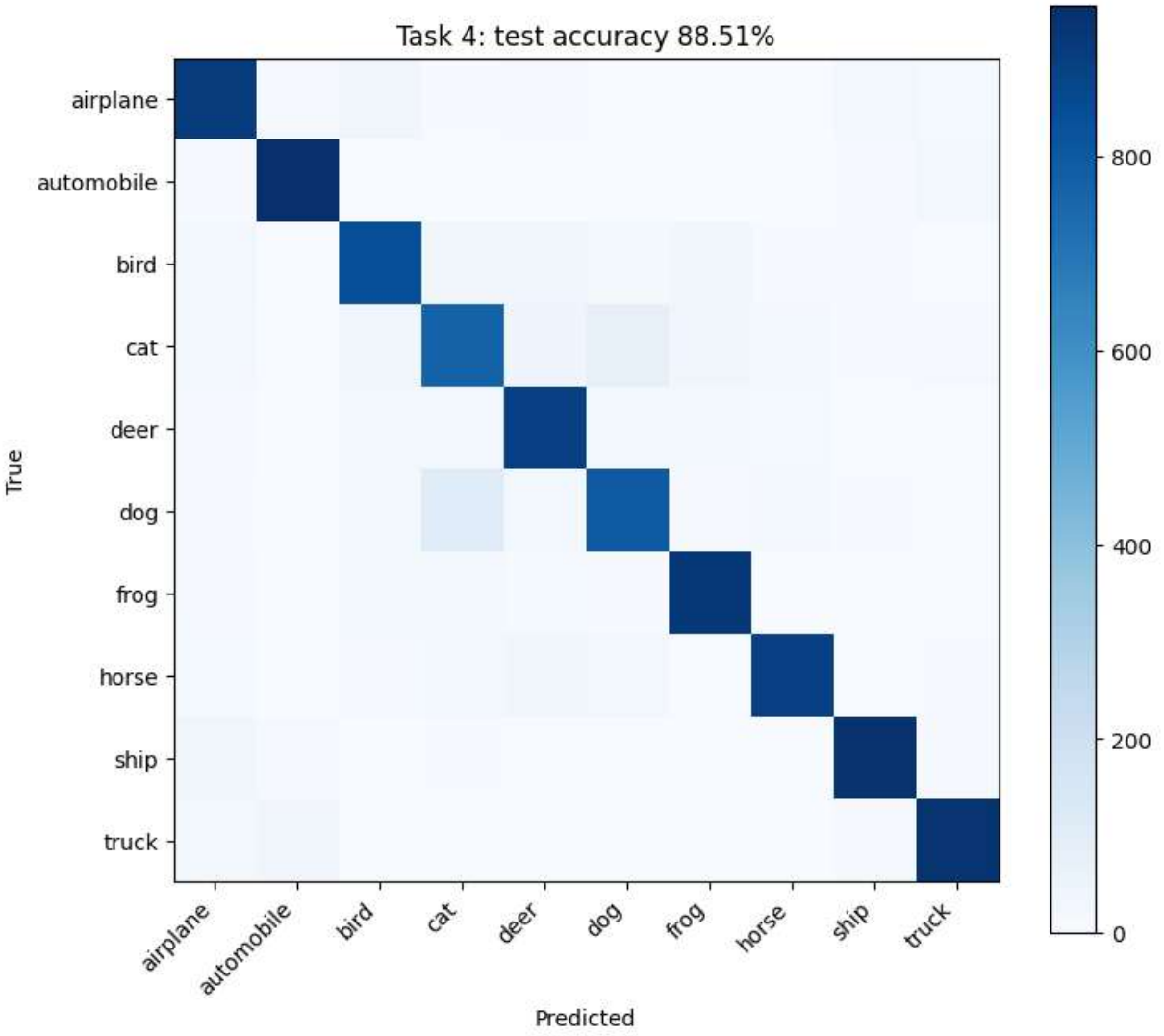
In [1]:
```
from pathlib import Path
import json
import types
import torch
from torch import nn
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
import matplotlib.pyplot as plt
import numpy as np
from IPython.display import display, HTML
from google.colab import drive
drive.mount('/content/drive')
OUTPUT_DIR = Path('/content/drive/MyDrive/EN4554_220381M/outputs')
DATA_DIR = Path('/content/data')
DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
checkpoint = OUTPUT_DIR / 'task4_full_a1a89e77b041.pt'
summary = checkpoint.with_suffix('.json')
if not summary.exists() or not checkpoint.exists():
    raise FileNotFoundError('Selected Task 4 checkpoint or result JSON missing; che
final_run = json.loads(summary.read_text())
assert final_run['status'] == 'complete'
```

```
final_name = 'full'
final_run['checkpoint'] = str(checkpoint)
config = final_run['config']
assert config['depth'] == 14 and config['widths'] == [24, 48, 96]
print('Restored selected full model; no training needed.')
print('Recorded final validation accuracy:', f"{final_run['history'][-1]['val_accur
eval_transform = transforms.Compose([transforms.ToTensor(),
    transforms.Normalize(config['mean'], config['std'])])
raw_train = types.SimpleNamespace(classes=['airplane','automobile','bird','cat','de
                                           'dog','frog','horse','ship','truck'])
def make_loader(dataset):
    return DataLoader(dataset, batch_size=128, shuffle=False, num_workers=0)
def atomic_json_save(payload, path):
    temporary = path.with_name(path.name + '.tmp')
    temporary.write_text(json.dumps(payload, indent=2))
    temporary.replace(path)
def show_table(rows, columns):
    from html import escape
    header = ''.join('<th>'+escape(str(c))+'</th>' for c in columns)
    body = ''.join('<tr>'+''.join('<td>'+escape(str(v))+'</td>' for v in r)+'</tr>'
    display(HTML('<table><tr>'+header+'</tr>'+body+'</table>'))

def initialize_network(model):
    for layer in model.modules():
        if isinstance(layer, nn.Conv2d):
            nn.init.kaiming_normal_(layer.weight, mode='fan_in', nonlinearity='relu
            if layer.bias is not None:
                nn.init.zeros_(layer.bias)
        elif isinstance(layer, nn.BatchNorm2d):
            nn.init.ones_(layer.weight)
            nn.init.zeros_(layer.bias)
        elif isinstance(layer, nn.Linear):
            nn.init.kaiming_normal_(layer.weight, nonlinearity='linear')
            nn.init.zeros_(layer.bias)

class CifarBlock(nn.Module):
    def __init__(self, in_channels, out_channels, stride=1, residual=True):
        super().__init__()
        self.conv1 = nn.Conv2d(in_channels, out_channels, 3, stride, 1, bias=False)
        self.bn1 = nn.BatchNorm2d(out_channels)
        self.conv2 = nn.Conv2d(out_channels, out_channels, 3, padding=1, bias=False
        self.bn2 = nn.BatchNorm2d(out_channels)
        self.residual = residual
        self.stride = stride
        self.out_channels = out_channels
    def forward(self, x):
        out = torch.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        if self.residual:
            shortcut = x[:, :, ::self.stride, ::self.stride]
            difference = self.out_channels - shortcut.shape[1]
            if difference:
                left = difference // 2
                shortcut = nn.functional.pad(shortcut, (0, 0, 0, 0, left, differenc
            out = out + shortcut
        return torch.relu(out)
```

```
class CifarNetwork(nn.Module):
    def __init__(self, depth=20, residual=True, widths=(16, 32, 64)):
        super().__init__()
        if depth < 8 or (depth-2) % 6:
            raise ValueError('Depth must equal 6n+2 with n >= 1')
        blocks_per_stage = (depth-2)//6
        self.stem = nn.Sequential(nn.Conv2d(3, widths[0], 3, padding=1, bias=False)
                                  nn.BatchNorm2d(widths[0]), nn.ReLU())
        blocks = []
        channels = widths[0]
        for stage, width in enumerate(widths):
            for block in range(blocks_per_stage):
                stride = 2 if stage > 0 and block == 0 else 1
                blocks.append(CifarBlock(channels, width, stride, residual))
                channels = width
        self.blocks = nn.Sequential(*blocks)
        self.head = nn.Sequential(nn.AdaptiveAvgPool2d(1), nn.Flatten(), nn.Linear(
        initialize_network(self)
    def forward(self, x):
        return self.head(self.blocks(self.stem(x)))

RUN_FINAL_TEST = True
TEST_RECORD = OUTPUT_DIR / 'task4_final_test_once.json'
if TEST_RECORD.exists():
    saved_test = json.loads(TEST_RECORD.read_text())
    print('Previously saved test result (test set not evaluated again):')
    print(json.dumps(saved_test, indent=2))
elif not RUN_FINAL_TEST:
    print('Test set untouched. Review Task 4, then set RUN_FINAL_TEST=True to evalu
else:
    config = final_run['config']
    final_model = CifarNetwork(config['depth'], config['residual'], tuple(config['w
    state = torch.load(final_run['checkpoint'], map_location=DEVICE, weights_only=T
    final_model.load_state_dict(state)
    assert sum(p.numel() for p in final_model.parameters()) <= 500_000
    assert config['epochs'] <= 20
    test_dataset = datasets.CIFAR10(root=DATA_DIR, train=False, download=True,
                                    transform=eval_transform)
    test_loader = make_loader(test_dataset)
    confusion = torch.zeros(10, 10, dtype=torch.int64)
    total_loss, total = 0.0, 0
    final_model.eval()
    with torch.no_grad():
        for x, y in test_loader:
            logits = final_model(x.to(DEVICE))
            total_loss += nn.functional.cross_entropy(logits, y.to(DEVICE), reducti
            predicted = logits.argmax(1).cpu()
            confusion += torch.bincount(y*10+predicted, minlength=100).reshape(10,
            total += len(y)
    saved_test = {'selected_variant': final_name, 'config': config,
                  'checkpoint': final_run['checkpoint'], 'test_loss': total_loss/to
                  'test_accuracy': confusion.diag().sum().item()/total,
                  'confusion_matrix': confusion.tolist()}
    atomic_json_save(saved_test, TEST_RECORD)
    print(f"Final test accuracy: {saved_test['test_accuracy']:.2%}")
```

```python
if TEST_RECORD.exists():
    saved_test = json.loads(TEST_RECORD.read_text())
    matrix = np.asarray(saved_test['confusion_matrix'])
    fig, ax = plt.subplots(figsize=(8, 7))
    image = ax.imshow(matrix, cmap='Blues')
    ax.set(xticks=range(10), yticks=range(10), xlabel='Predicted', ylabel='True',
           title=f"Task 4: test accuracy {saved_test['test_accuracy']:.2%}")
    ax.set_xticklabels(raw_train.classes, rotation=45, ha='right')
    ax.set_yticklabels(raw_train.classes)
    fig.colorbar(image, ax=ax)
    plt.tight_layout()
    plt.show()
    show_table([[name, f'{100*matrix[i,i]/matrix[i].sum():.2f}']
                for i, name in enumerate(raw_train.classes)], ['Class', 'Test accur
```

```
Mounted at /content/drive
Restored selected full model; no training needed.
Recorded final validation accuracy: 88.88%
100%|██████████| 170M/170M [52:51<00:00, 53.8kB/s]
Final test accuracy: 88.51%
```



Class	Test accuracy (%)
airplane	90.90
automobile	95.40
bird	83.90
cat	77.00
deer	89.20
dog	80.00
frog	92.20
horse	89.30
ship	94.00
truck	93.20

Final-test interpretation: The selected model achieves **88.51% test accuracy**, close to its **88.88% validation accuracy** (a 0.37-point difference). This suggests similar performance on unseen images. Automobile is the strongest class (**95.40%**); cat (**77.00%**) and dog (**80.00%**) are the hardest. The confusion-matrix diagonal contains correct predictions; off-diagonal entries show mistakes. The saved test result is reused to avoid another evaluation.

Overall, the model correctly classifies 8,851 of the 10,000 test images. The lower cat and dog accuracies show that performance is not equally strong for all classes. The per-class table helps identify these weaknesses even when overall accuracy looks good. This is the final evaluation of the validation-selected checkpoint; it should not be used to tune further settings.

Conclusion

Task	Completed work	Main result and lesson
1: Backpropagation	Manual layers, two gradient checks, MLP training, and activation comparison	Checks pass; MLP reaches **51.22% validation accuracy**. Sigmoid shows vanishing gradients in the initialization test.
2: Optimizers	Custom momentum and Adam, trajectory plots, and nine CNN runs	Custom updates match PyTorch. Momentum at 0.02 gives the best tested CNN result: **71.50% validation accuracy**.
3(a): Regularization	Seven settings on 5,000 training images, accuracy curves, and gap comparison	Flip/crop performs best at **57.24% validation accuracy**. More regularization is not always better.

Task	Completed work	Main result and lesson
3(b): Network depth	Plain/residual pairs at depths 8, 20, and 56	Residual-56 reaches **90.90% validation accuracy**; shortcuts make the deeper model much easier to optimize.
4: Final CNN	Three design ablations, validation selection, and final test evaluation	The **386,626-parameter** model trains for **20 epochs** and achieves **88.51% test accuracy**.

All assignment tasks, including optional Task 4, have recorded results. The experiments show that correct gradients, a suitable learning rate, balanced regularization, and residual shortcuts each affect learning. The final model meets the size and training limits, with test accuracy close to its validation accuracy of 88.88%.

The experiments use one fixed seed, so small differences should be interpreted cautiously. Comparisons across tasks use different models, data amounts, or training budgets and are not controlled rankings of every method.